EurekaZero

Engineered Intelligence for Algorithm Invention by Self-Play

JopeDime Team

Draft — October 2026

Abstract

[TBD: Supply the problem statement, proposed method, supported task scope, contributions and evaluation summary from Jepson's source.]

[TBD: State the headline first-proposal result with its uncertainty once the independent evaluation is complete.]

1 Can Machines Invent?

We specify how to test whether feedback from execution improves the first algorithm a system proposes for a new problem. AlphaGo Zero supplies a precedent within a fixed game: it went beyond the best human play without seeing a human game, learning from self-play under the rules of Go [101].

The skeptics' challenge. Language models retrieve existing solutions rather than find new ones, Yann LeCun argued in March 2025: "in their purest form, they are incapable of inventing new things" [58]. Thomas Wolf warned that current trends will give us "a country of yes-men on servers" [117].

Dwarkesh Patel had asked Dario Amodei in August 2023 what to make of models with "basically the entire corpus of human knowledge memorized" that had not made "a single new connection that has led to a discovery" [80]. In January 2025, Demis Hassabis posed the hardest version as a benchmark for general intelligence: today's systems "can play a game of Go at a world champion level. But could a system invent Go?" [41]. These challenges concern both proposing solutions and choosing new questions; our test concerns only algorithm proposals under human-defined tasks and metrics (Section 8).

Predicting historical text does not directly reward improvement over an incumbent. That motivates a different training signal; it does not establish a ceiling on what a pretrained model can discover (Section 2).

The human challenge. Research effort rose while productivity declined in the industries, products and firms examined in the 2020 study by Bloom et al. [8]. Their Moore's law example required more than 18 times as many researchers as in the early 1970s to sustain chip-density growth. Jones [46] found later first inventions and greater specialization in the inventor data he studied. Park et al. [79] reported declining disruption on citation-based measures of papers and patents. These observations motivate our question: can execution feedback help a proposer depart usefully from familiar ideas? Section 2.4 develops the role of the scientific record.

Key idea

Can judgment bought during training improve the first proposal at deployment?

From search to learning. Automated research systems already couple proposals to tests [37, 49, 65, 75, 104, 105]. Sections 2.5 and 3 compare how they generate, judge and retain useful work.

Keeping a winning program preserves an output. Training a proposer on those outcomes may preserve a capability that transfers to a new problem.

The Zero move. AlphaGo learned from human expert games and searched heavily at every move [100]. AlphaGo Zero learned only from games against itself, and its network alone, with no search, rated 3,055 Elo. That is close to the 3,144 of AlphaGo Fan, the version that beat the European champion Fan Hui [100, 101].

This motivates an analogous goal for invention: learn from repeated program evaluations during training, then deploy a writer with a single-candidate budget, search = 1 (Box 1). The budget is a test condition, not a promise that every first proposal wins. Figure 1 separates the training feedback path from the frozen writer and fitting procedure at deployment.

Engineered intelligence. The reference design makes each of the four pillars in Section 2 an inspectable component of EurekaZero: aim, variation, judgment and retention. The aim is a declared task and metric. Variation comes from a grammar-constrained program writer. Judgment is a referee that scores each program on folds withheld from its fit. Retention uses two ratchets. A board's incumbent changes only on strict improvement. A retrained writer replaces the serving one only after a held-out admission test (Table 1). We call this *engineered intelligence*: a capability whose aim, judge and memory are designed to be audited, rather than left implicit in a pretrained model. Pretrained components still supply prior

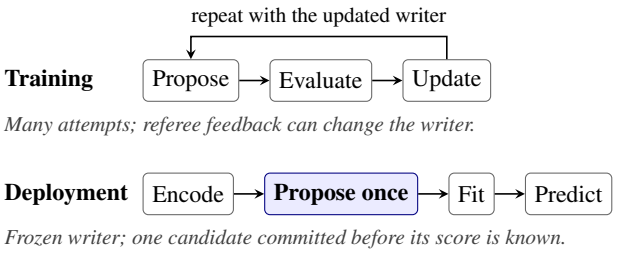


Figure 1: **Training feedback and the single-candidate contract.** Schematic of the reference procedure. The writer updates during training; the emitted estimator fits during deployment. Whether the learned writer improves the first proposal is the empirical question in Section 7.

knowledge (Box 1). Engineering fixes what the system is rewarded for and how its improvement is checked.

Box 1: What "search = 1" means

- **One inference, one candidate.** For a new problem, the program writer emits one complete program ($B = 1$), with no retries, repairs, best-of-N, reranking or weight updates.
- **Everything is counted,** including reasoning tokens and problem encoding. One inference is one generation, not one forward pass.
- **Declare the fitting procedure.** Fitting the emitted algorithm to training rows is allowed. Any internal model or hyperparameter selection must be disclosed and counted in its compute budget. One outer proposal does not establish the absence of internal search. Training search is priced separately in Section 5.

A single candidate, with prior knowledge: human-designed program primitives and pretrained encoding components supply prior knowledge. The Zero analogy concerns the additional training feedback from the referee.

Contributions. We formulate algorithm invention as a game and design a learner for it, instantiated for tabular prediction tasks whose moves are programs in a typed language of learning primitives.

1. **The anatomy of innovation:** four pillars (aim, variation, judgment and retention), with an idealized relation between proposal quality and the number of attempts (Section 2).
2. **The innovation game:** a board, move, referee and opponent, with a reward ordering and two ratchets, that make invention playable on many boards (Section 4).
3. **The economics of algorithmic self-play:** a cost model for training and for the cost per verified improvement (Section 5).
4. **EurekaZero:** a reference learner with a frozen problem encoder, a grammar-constrained program writer and a separate problem–program map, plus an expert-iteration procedure and safeguards against reward hacking (Section 6).
5. **An evaluation protocol for the single-proposal claim:** paired comparisons with frozen algorithms whose receipts record the model that designed them, performance curves against agents allowed repeated attempts, and a test of the map's ranking against a program-only control (Section 7).

Two stages of development. The offline multi-dataset reference pairs a neural program writer with a problem–program ranking map. Search-scored lists train both; native neural proposals seed further search. A second stage studies a closed-loop admission ratchet. Its inspected implementation proposes programs through heuristic edits, retrieval and grammar sampling, and updates only a ranking adapter (Section 6.8). The retained admission and program-event logs concern this second task; binding them to that source remains pending. They do not establish a handoff from the offline writer and map. First-proposal quality on unseen families and single-candidate deployment remain unqualified. Section 7 specifies the independent comparisons required before claiming an advantage.

Section 2 separates innovation's roles. We trace their AI building blocks (Section 3), then define the game and learner. The outcomes of those comparisons will bound the claims in Section 8.

2 The Anatomy of Innovation

We organize innovation into four roles: aim, variation, judgment and retention. Together they connect a proposal, its evaluation and what the next attempt retains. We call judgment learned before a try *taste*, and ask whether it can improve the first proposal.

2.1 Novelty is free

Novelty is the cheapest part of invention. Borges imagined a library of every possible book of 410 pages [11]. Every algorithm short enough to fit in a volume is already on its shelves. Nothing in the library tells a reader which volume is true. A random string generator is the same library with a faster librarian: almost every string it emits is new, and almost none is worth running (Figure 2, lower right).

Language-model studies test this distinction. In a blinded study with 104 NLP researchers, ideas from a language model were rated more original than experts' ideas ($p < 0.05$) [98]. Once 43 experts had spent over 100 hours each carrying out ideas from both pools, the generated ideas lost significantly more ground on every metric [99].

Boden had predicted the asymmetry in 1998: machines, she wrote, would find generating new ideas easier than automating their evaluation [10]. These studies motivate a distinction between the novelty of a proposal and its value after execution; they do not establish that judgment is the only bottleneck in every domain.

2.2 The four pillars

Accounts of discovery from biology, psychology, philosophy and engineering provide a useful common vocabulary

Table 1: **A four-part reading of traditions of discovery.** A dash marks a pillar outside a tradition's scope: Wallas analyzed a single act of thought. For Darwin and Campbell, the aim is a criterion set by the environment. Engineering is the design–build–test–learn cycle [74]; AlphaGo Zero follows Silver et al. [101]; EurekaZero is the reference design (Sections 4 and 6).

Tradition	Aim	Variation	Judgment	Retention
Darwin [25]	Fitness to the environment	Variation	Natural selection	Inheritance
Wallas [115]	Preparation	Incubation, illumination	Verification	—
Campbell [15]	A selection criterion	Blind variation	Selection	Retention
Popper [87]	The problem	Conjecture	Refutation	The surviving theory
The scientific method	Question	Hypothesis	Experiment	Publication
Engineering	Design target	Design and build	Test	Learn
AlphaGo Zero	Win the game	Moves chosen with search	Game outcome	Network weights
EurekaZero	Problem encoding and metric	Program writer	Referee on hidden folds	Incumbent pool and weights

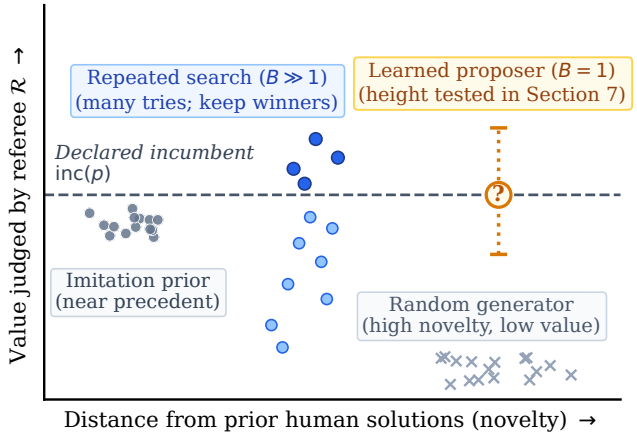


Figure 2: **Novelty is free; judgment is the inventor.** Schematic. Horizontal: distance from what people have already done. Vertical: value as judged by the referee. A random generator travels far and finds little. Imitation, repeated search and a learned proposer illustrate different ways to select candidates; their positions are conceptual, not measured limits. The value of EurekaZero's single proposal remains an empirical question for Section 7.

(Table 1). The four-pillar organization is our synthesis of these traditions, rather than a claim that their theories are identical.

Aim fixes the problem and what counts as better. Hamming asked colleagues, "What are the important problems of your field?" [40]. Pólya's first step is to understand the problem [85], and Popper argued that observation is always selective, so it presupposes a problem [87].

Variation proposes candidates. Peirce called abduction "the only logical operation which introduces any new idea" [81, CP 5.171]. Campbell modeled creative thought as blind variation [15, 103]. Koestler located creative acts in bisociation, joining two unrelated frames of thought [52]. Boden named three routes to a new idea: combining familiar ideas, exploring a conceptual space and transforming it [10].

Judgment tests a candidate against something outside the mind that proposed it: the attempted refutation [86], the experiment, peer review, and the referee of the common task framework, which scores every entry on data the competitors never see [29].

Retention preserves what survives: inheritance for Darwin [25], selective retention for Campbell [15], the textbook for a science and the incumbent for a game. It can store either an output or a way to propose better outputs. Archiving a winning program does the first; using evaluation feedback to update the generator attempts the second. This distinction connects the four pillars to the learning question in Section 1.

Campbell argued that every genuine gain in knowledge needs a mechanism for introducing variation, a consistent selection process, and a mechanism for preserving and reproducing what is selected [15, p. 381]. The aim is what keeps the selection consistent, which makes it the fourth pillar.

2.3 Taste is judgment moved upstream

Skill in invention is judgment that arrives before the try. For Poincaré, creating meant not making the useless combinations: "Invention is discernment, choice." The sterile combinations, he added, do not even present themselves to the inventor [83, pp. 324–325].

Chess masters show the same economy: in de Groot's think-aloud studies they considered about as many moves as weaker players but found the right ones [26]. Chase and Simon studied chunks through perception and recall tasks [18]. Much of that knowledge is tacit; experts know more than they can say [84].

AlphaGo's policy network learned to select moves from human expert games, then refined that judgment through self-play [100].

Taste has a price we can state. Suppose each try beats the incumbent with a fixed, nonzero probability q, independently of the others. The number of tries up to and including the first win is then geometric, with mean

$$\mathbb{E}[N_{\text{tries}}] = \frac{1}{q}, \tag{1}$$

where N_{tries} counts all candidate attempts, including incomplete or invalid outputs and failed executions. Under this

fixed-probability, independent-try model, better proposal quality shortens the expected search.[1]

The equation isolates the value of learning before deployment. It does not rank random generators, pretrained models and agent systems without measuring their proposal quality. In particular, search $= 1$ fixes the number of candidates a system may submit; it does not require the probability of a win to approach certainty. A single-candidate system can fail, and every failure must remain in its reported results. Section 5 uses the same idealization to relate evaluation cost to the cost of a verified improvement.

Key idea

Novelty is free. Judgment is the inventor.

2.4 Learning from a referee

People learn much of their taste from the record of their field. Normal science, in Kuhn's account, solves puzzles inside a reigning paradigm [55]. The record also holds people in place: when a star life scientist dies prematurely, articles by outsiders to the star's collaborators flow into the field, draw on different literature and are disproportionately likely to be highly cited [2].

A model trained to predict text receives a signal about what was written, rather than a direct score for improving an algorithm. That does not prevent novel generalization. It motivates supplementing the historical record with feedback from execution, so a useful departure from precedent can be rewarded.

A referee offers such an additional teacher. AlphaGo Zero learned with no human data, from the rules and the outcomes of its own games, and found strategies outside traditional Go knowledge [101]. Its predecessor had already made such a move. As David Silver told Wired in March 2016, AlphaGo put the chance that a human would play its Move 37 against Lee Sedol at one in ten thousand, and it won that game [69].

The knowledge then flowed back to people. In more than 5.8 million moves by professionals from 1950 to 2021, previously unseen moves grew more frequent once superhuman programs arrived, and came with better decisions [97]. These results establish the possibility in Go. Whether an analogous signal improves algorithm invention must be measured in the innovation game.

2.5 Where the pillars live

Machines do not remove the pillars; they move them (Table 2). The most ambitious swarm design fits the same anatomy. In an October 2026 essay drawn from his book, Socher rests the Eureka Machine on "a living map of all human knowledge", "a unified model of physical reality", "high-fidelity digital twins" and "fully autonomous physical laboratories" [104, 105]. On top of them, millions of AI agents critique each other's ideas.

[1] Adaptive search and correlated proposals need not follow Equation (1); independence alone does not say whether the approximation overstates or understates their cost. Empirical attempt curves are required for those systems.

Table 2: **Where feedback can be retained.** The agent-search column describes fixed-weight deployment loops, not every agent system. The EurekaZero column states the proposed training and deployment protocol. In every column, people still set the aim.

	Human science	Agent search	EurekaZero
Aim	The researcher chooses	A person writes the goal	Problem encoding and metric, chosen by people for now
Variation	The scientist's ideas	Agents propose and critique	One candidate per test problem
Judgment	Experiments, peer review	Experiments, simulators, autonomous labs	The referee, on hidden folds
Retention	The literature	Logs, archives, a map of knowledge	Incumbent pool and the writer's weights
Where taste lives	Years of training, in one head	Weights fixed; context can adapt	Training on referee feedback
Deployment budget	Varies by problem	Multiple adaptive attempts	One candidate; success measured

In our terms, the map is retention; the model of reality, the twins and the laboratories are faster and better judgment; and the agents supply variation and critique.

Agent search can retain programs, revise its context or update its proposer. The label *swarm* does not specify which it does. The comparison in Table 2 isolates deployment with fixed proposer weights; EurekaZero aims to improve the proposer through training before a new problem arrives. Both need a reliable referee, and training costs belong in any account of deployment savings.

The next section traces the methods that connect these four roles (Section 3).

3 The Road to Zero: Five Building Blocks

Five lines of AI research supply the building blocks for the four pillars of Section 2: deep learning, program search, reinforcement learning, games and the race for self-improvement.

The question is whether their combination can turn experience across algorithm-discovery tasks into a better first complete program for a new family.[2]

3.1 Deep learning gave variation a voice

Deep learning built the variation pillar: machines that propose. Models learned representations from data [59]. AlexNet made the case on images [54], the Transformer modeled sequences with attention [112], and empirical scaling laws related language model loss to model size, dataset size and training compute [42, 48]. Sutton [108] drew the lesson: general methods that use computation win.

Generative models turned representations into proposals in text, video [14] and code. Codex solved 28.8% of the HumanEval programming problems with one sample, and

[2] Each history keeps only the milestones the argument needs; Appendix A collects the referees those milestones rely on.

70.2% with 100 samples, counting any that passed the unit tests [19]. That gap illustrates the value of a larger sampling budget. It is not a direct estimate of the geometric model in Equation (1): pass rates and the probability of beating an incumbent measure different outcomes.

Learned prediction and selection also support scientific discovery: protein structures that rival experiment in most CASP14 cases [47], 2.2 million crystal structures predicted stable relative to the Materials Project [68], and halicin, an antibiotic that then treated infections in mice [106].

CLIP put images and captions in one shared space [88], an analogy for our problem–program ranking map (Section 6). The reference map ranks scored programs within each problem; it does not condition or screen the writer. Its ranks can influence training through search-parent selection (Section 6.4). Ranking accuracy and first-proposal quality therefore need separate tests.

EurekaZero draws on these three ingredients: representations that read a problem, models that write code and a separate ranking map. What remains is training a proposer directly on verified algorithm quality and testing transfer beyond its training tasks.

3.2 Program search built the variation–judgment loop

Program evolution joined variation to judgment: propose a program, score it, keep the better ones. Friedberg [35] let a machine vary its own instructions, Holland [43] made variation and selection a general method of adaptation, and Lenat's AM and EURISKO searched concepts and heuristics by measures of interest [63, 64]. EURISKO takes its name from the Greek verb behind *eureka*, as ours does. Koza [53] closed the loop with genetic programming: a population of programs, a fitness function as referee and selection as coach. Populations retain programs; learned proposers can also retain patterns for later tasks.

Later work sharpened both halves: superoptimization [67], program synthesis from examples [38] guided by neural networks [5], neural architecture search [124], and AutoML-Zero, which evolved learning algorithms from basic mathematical operations [89].

Then language models became the variation operator. FunSearch found cap-set constructions beyond the best known [91]: verified new solutions within a fixed task, bought with search. Lion, an optimizer found by program search, went into production at Google [20]. AlphaEvolve multiplied two 4×4 complex matrices with 48 scalar multiplications, improving on Strassen's algorithm in that setting after 56 years [75]. MLEvolve carries the loop into machine-learning engineering [30].

EurekaZero adopts executable programs as moves inside this generate–judge loop, and asks how much of a verified search can transfer into the proposer itself. DreamCoder already learns reusable libraries and a search-guiding model across tasks [31]; a single-candidate evaluation tests how useful a learned proposal is before additional task-specific search.

Box 2: Self-play in 90 seconds

A player starts with only the rules and plays itself. A short search before each move finds better moves than instinct would, and the referee scores each finished game. The network learns to propose the moves the search chose (its policy) and to predict the winner (its value), so the next search is stronger. Over millions of games, search is distilled into intuition [1, 101]. *What if the board were a problem, and the move an invention?*

3.3 Reinforcement learning turned retention into taste

Reinforcement learning turns a referee's verdicts into a better player. Bellman [6] gave it an equation of value. Samuel's checkers program improved by playing a copy of itself [93], TD-Gammon reached near the best human backgammon players by self-play [109], and DQN learned Atari games from pixels [70]. AlphaGo learned from human games and searched at every move [100]; AlphaGo Zero learned by self-play alone [101]. AlphaZero mastered three games with one algorithm [102], MuZero learned a model for planning [95], and AlphaStar's league of mutually exploiting agents reached Grandmaster level in StarCraft II [113].

In expert iteration [1], search is the teacher and the network is the student (Box 2). Language models also learn from feedback: human preferences [22, 78] and rule-based checks of answers, as in DeepSeek-R1 [39].

From this line of work, EurekaZero takes training on referee-scored play and the lesson from Go that a network can absorb search; the remaining challenge is defining a reward for relative algorithm quality and separating gains on training games from transfer to unseen families.

3.4 Gamification made judgment cheap

Games built the judgment pillar at scale. Shannon [96] chose chess because its moves and its goal are sharply defined. The common task framework made machine-learning research a game: shared data, enrolled competitors and a referee who scores every entry on a sealed test set [29], as in the Netflix Prize [7] and ImageNet [92]. People played science too. Foldit players beat automated methods on hard protein puzzles [23] and solved a protease structure that earlier attempts had not [50]; EteRNA players, whose designs were tested in a laboratory, found RNA design rules [60].

Then machines played problems. AlphaTensor's TensorGame beat the best known matrix-multiplication algorithms for several sizes [33], and AlphaDev's AssemblyGame found sorting routines now in the LLVM C++ library [66]. AlphaGeometry learned olympiad geometry without human demonstrations [110]. AlphaProof, using the Lean proof checker, and AlphaGeometry 2 jointly reached silver-medal standard on the 2024 International Mathematical Olympiad problems, using computation over multiple days [45].

These benchmarks supply the recipe for a game (clear rules, a referee, a score and a sealed test) and raise the question of

Table 3: **Related approaches retain experience in different forms.** This qualitative comparison separates output, deployment budget and retained state. Exact budgets depend on each study's protocol. The last row states EurekaZero's design; its proposal quality is still to be tested.

System	Output	Deployment budget	Retained experience	Feedback source
Genetic programming; AutoML-Zero	An algorithm	Study-specific search	Selected programs	Executable objective
AlphaTensor; AlphaDev	An algorithm for one family	Task-specific search	A policy for that game	Game reward
FunSearch; AlphaEvolve	An algorithm	Iterative search	Program archive	Automated evaluation
Research swarms (AI Scientist, autoresearch)	Experiments and code	Adaptive attempts	Context, logs and archives	Experimental outcomes
EvoTune; TTT-Discover; Evolution Fine-Tuning	An algorithm or construction	Protocol-specific search	Fine-tuned policy or search state	Search trajectories
TabPFN; MotherNet; KumoRFM	Predictions or network weights	Task-specific inference	Pretrained parameters	Training across tasks
Absolute Zero; STP	Answers and proofs	Decoding budget varies	Policy parameters	Correctness feedback
EurekaZero	An algorithm	One candidate	Proposed writer updates	Referee scores ranked within each problem

which parts of a task can vary under a shared representation and referee. EurekaZero makes the problem an input to the proposer (Section 4); whether that supports transfer across families remains a result to establish.

3.5 The race for self-improvement scaled search

The race for self-improvement spans program search, agent scaffolds and learned policies. Good [36] foresaw an "intelligence explosion", and Schmidhuber [94] specified a Gödel machine that rewrites its own code once it proves the rewrite useful.

STOP improves the scaffold that calls a language model, including itself, and its authors note: "Since the language models themselves are not altered, this is not full recursive self-improvement" [120]. The Darwin Gödel Machine's self-editing agents raised its SWE-bench score from 20.0% to 50.0% [121], and AlphaEvolve cut Gemini's training time by 1% [75].

Language models also play themselves. SPIN plays its previous iteration [21], Absolute Zero proposes and solves coding tasks under a code executor [122], and STP pairs a conjecturer with a Lean prover [28]. Bailey et al. [3] attribute plateaus in LLM self-play to reward hacking by the conjecturer and evaluate self-guidance in formal theorem proving. Our closest neighbors learn to discover. EvoTune fine-tunes on what its own evolutionary search finds [107], and TTT-Discover trains on the one problem at hand, at a few hundred dollars per problem [119]. Evolution Fine-Tuning learns from 156,000 search trajectories across 371 tasks [61], and DiscoRL meta-learned a reinforcement-learning rule that beat human-designed ones [77]. These approaches retain improvements in different places: a search trajectory, a task-specific learner or a reusable learning rule. Our comparison asks about the quality and cost of the first complete candidate, rather than treating these forms of learning as absent.

Research itself is becoming a swarm. The AI Scientist automates ideas, experiments and papers [65], and the AI co-scientist ranks agents' hypotheses in an Elo tournament [37]. Karpathy's autoresearch, released in March 2026, runs about 100 experiments overnight and keeps what improves [49]. In May 2026, Recursive raised $650 million to build AI that improves itself [13]. In the innovation game, an encoded research task would be the board and its evaluator the referee (Section 4).

Self-improvement comes in three strengths: a system can find better external algorithms, build better components of itself, or improve its own ability to improve, raising verified discoveries per unit of compute across generations. Each requires separate evidence (Section 8); a recorded training trajectory alone establishes none of them on an unseen task.

From these systems EurekaZero keeps both the destination and the safety frame, verifying every improvement before keeping it and sandboxing every program, while testing whether learning across discovery episodes improves the first proposal over the corresponding frozen proposer under the same deployment budget.

3.6 Where the roads meet

Table 3 distinguishes what a system produces from how it retains experience. TabPFN, MotherNet and KumoRFM provide neighboring examples of learning across tasks [34, 44, 73]. Their prediction or network-generation interfaces differ from the proposed readable-program output of EurekaZero, but a common candidate budget alone would not make their computation or task assumptions equivalent. Absolute Zero and STP learn by self-play, with correctness as feedback. Our proposed comparison measures improvement against a declared incumbent. That standard supplies the opponent; the next section specifies the board, program, referee and promotion rules needed to judge a move (Section 4).

Key idea

The test is whether experience on earlier discovery tasks improves the first complete proposal on a new one.

4 Innovation Is a Game

We define one attempt at algorithm invention as a game: the encoded problem is the board, a program is the move, the referee checks and scores it, and a declared incumbent is the opponent.

Encoding the problem provides a shared interface; each task type still needs its own qualified encoding, language and evaluation protocol (Section 4.5).

4.1 Why games came first

Games became AI's laboratory because they supply four things at once: fixed rules, an impartial referee, a clear winner and cheap replay. Chess was chosen because it is "sharply defined both in allowed operations (the moves) and in the ultimate goal (checkmate)" [96], and checkers showed that a program could improve by playing itself [93].

Together these properties support repeated, reproducible evaluation. People scale judgment the same way, through prizes, leaderboards and the common task framework's "scoring referee" [29] (Section 3). Invention has the first three properties once the problem and the solution are written down, within the limits of Section 4.6; cheap replay is a question of cost (Section 5).

4.2 The rules

Table 4 sets the game beside Go. An illegal move is counted as a failed attempt. The reference writer reads the encoded problem. The referee keeps the incumbent as a separate comparison standard (Section 6.2); its code and score must stay outside the writer's input.

Formally, the game is a tuple of seven parts,

$$\mathscr{G} = (\mathscr{P}, \phi_{\mathrm{P}}, \mathscr{S}, \phi_{\mathrm{S}}, \mathscr{R}, r, \mathscr{O}).$$

Here ϕ_{P} and ϕ_{S} embed a problem $p \in \mathscr{P}$ and a program $w \in \mathscr{S}$ as z_p and z_w. The referee $\mathscr{R}$ reports a program's validity and score, the reward r orders moves, and the opponent $\mathscr{O}$ supplies the incumbent $\mathrm{inc}(p)$. The tuple extends Rice's model of algorithm selection [90]: the player writes the algorithm instead of picking it, and a declared incumbent sets the comparison standard.

With one move, the game is a contextual bandit: the board supplies context, the program is the action and the verdict is the reward [57]. It earns the name through an opponent that improves as the player does. The asymmetry is in the moves: Go offers about 250 legal moves per turn over about 150 turns, each free to check [100], while this game offers one move, costly to judge. Under a single-attempt budget, the chance q of a win matters directly (Equation (1)). Whether training raises that chance on new families is the test in Section 7.

Table 4: **Go and the innovation game, element by element.** Moves differ most: Go's are cheap, while each program is costly to judge (Section 4.6). Go's game length is from Silver et al. [100].

	Go	The innovation game
Board	Stones on a 19×19 grid	The encoded problem
Move	Place one stone, or pass	Write one complete program
Legal move	Empty point; no suicide or forbidden repetition	Parses, type-checks, runs within limits
Illegal move	Not allowed	Stillborn: an automatic loss
Referee	The rules of Go	The verifier, on hidden folds
Win	Higher final score	Valid, and beats the incumbent
Opponent	A past self	The incumbent, often a past self
Rating	Elo	Win rate and Elo across problems
Moves per game	About 150, each cheap	One, costly to judge

Figure 3 separates training from game day. Its top row follows the single-candidate contract in Box 1.

4.3 The board and the move

The board is the aim pillar of Section 2, made machine-readable: a summary of the training data, the objective and metric, and the constraints. Landmark scores from cheap learners [82] and learned tabular summaries are possible encodings; Section 6.2 specifies the reference encoder and its cost limit.

A useful board should preserve distinctions relevant to algorithm choice, remain stable under irrelevant reorderings, exclude final evaluation information and have a measured cost. These are requirements to test, not properties guaranteed by naming an encoder. People still choose the problem and define what counts as a useful solution.

A move is one program in a typed language over learning primitives. A task-specific grammar restricts legal moves; Section 6.3 specifies the reference vocabulary and composition limits.

Types remove whole classes of stillborn programs before they run [71]. Canonicalization can merge syntactic variants; it need not identify every functionally equivalent implementation. AutoML-Zero reused scores when hashes of truncated predictions matched after a short training and validation probe [89]. Matching a probe does not establish agreement on other inputs (Section 6.3).

The language bounds what can be expressed; changing its primitives changes that space [9]. The reference grammar also bounds composition depth. Structural constraints can prevent invalid programs; their effect on useful proposal quality must be measured (Section 7).

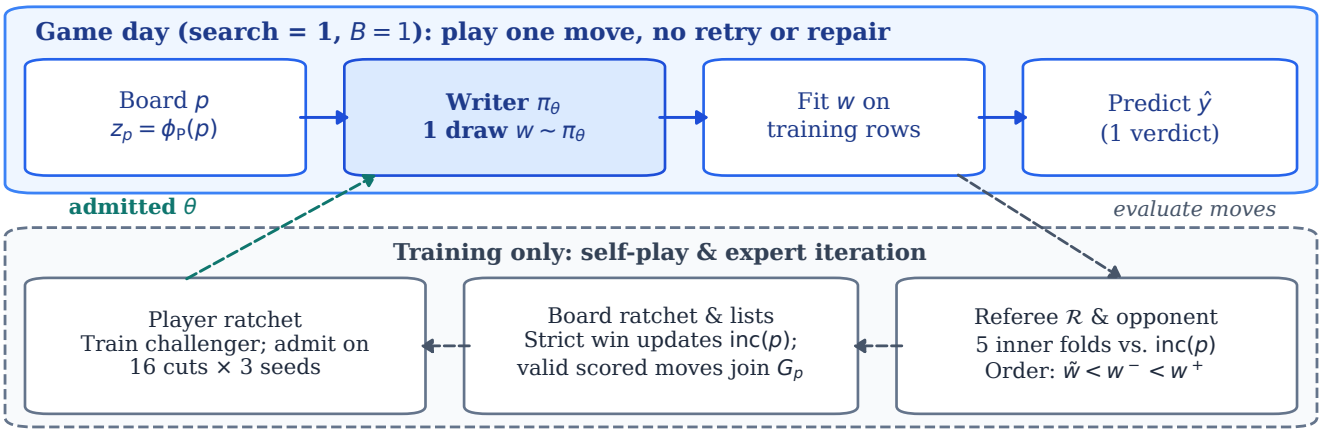


Figure 3: **Training plays the whole game; game day plays one move.** Schematic. Top row (blue): game day. Dashed box: training only. Outcomes are recorded; valid scored programs can enter the learning lists, while failed attempts still count against the budget.

4.4 The referee and the opponent

The reward is an ordering first and a number second. Validity is a gate: any stillborn program scores below any valid one. Past the gate, programs rank by improvement over the incumbent on hidden folds. The reference promotion rule keeps the existing incumbent on a tie. Novelty never pays for losing: any novelty bonus (Section 5) must respect the order. For a stillborn program $\tilde{w}$, an also-ran w^- and a win w^+ on board p,

$$r(p,\tilde{w}) < r(p,w^-) < r(p,w^+).$$

The opponent is a ratcheting incumbent. A program must strictly improve under the same evaluation protocol to displace a board's incumbent. A retrained writer faces a separate admission test (Section 6.5). Together the two ratchets make this self-play: the player keeps facing a standard often set by its own earlier play. AlphaGo Zero likewise separated network admission from choosing a move during play [101].

Subtracting the incumbent's score does not change which move is best. Promotion raises the bar, turns yesterday's winners into today's also-rans and shifts which problems are worth playing. The reference learning targets use each program's rank within its scored list (Sections 6.4 and 6.5).

4.5 One game, many boards

The accepted checkpoint route and both fixed comparators support only tabular regression. Binary and multiclass classification, ranking, forecasting and relational prediction remain prospective boards, each needing a qualified encoding, program language, metric and validation protocol.

Beyond machine learning, examples of referees include sorting tests [66], evaluators for mathematical constructions [91] and accelerator-kernel timers [75]. These motivate extensions of the recipe, rather than results of this paper (Section 8).

Casting algorithm discovery as a game is not new, but earlier games wrote one family of problems into their rules, as AlphaTensor did for matrix multiplication and AlphaDev for sorting [33, 66]. AlphaZero used one algorithm for three games and trained a separate network for each [102]. Here the problem is an input on the board, so one player can, in principle, learn many boards.

Key idea

A reliable referee makes a discipline a candidate for this game; transfer to that discipline still needs evidence.

4.6 The analogy's limits

A worked game, chosen by a rule fixed before inspection, is to be reported with the results (Section 7.6). The analogy breaks in four places.

The referee is noisy: picking a winner by a noisy validation score biases that score upward [17]. Strict numerical improvement alone does not quantify confidence; independent evaluation and an uncertainty protocol are needed to interpret it.

Moves are expensive, each a fit and a cross-validation (Section 5).

Judges can be gamed: the evaluation harness must resist exploits [114]. Section 6.6 specifies the isolation and parity checks required here.

And **no algorithm wins everywhere**: averaged uniformly over all problems, no learner has lower error off the training set than another, ours included [118].

Real problems are structured, though, and a player that reads the board can choose its bias board by board. Section 7 reserves the tests of that hypothesis against fixed frontier-designed regressors, with paired outcomes beside Elo summaries [32]. Non-transitive comparisons are one reason not to reduce the evidence to a single rating [4].

Those rules define an attempt. Pricing a verified improvement must also count attempts that fail or lose (Section 5).

5 The Price of Play

A verified improvement carries the cost of the attempts that precede it. The ledger includes encoding, input tokens, generation, repairs, referee execution, updates and operating costs.

The planning ratios below organize that bill; a full-run price requires the measurements in Sections 5.6 and 7.

5.1 The bill in one line

For independent attempts with fixed win probability and cost distribution, expected cost per win is mean cost per attempt divided by that probability (Equation (1)):

$$C_{\text{win}} = \frac{C_{\text{attempt}}}{q}. \tag{2}$$

C_{attempt} is the mean incurred cost per candidate attempt, including failures. A repair that emits a revised candidate is another attempt; charge its work once. q is the probability that an attempt beats the incumbent.

It factors as $q = q_{\text{run}}\, q_{\text{beat}}$, the share of moves that run times the share of those that win. Valid losses still cost money, so divide by q, not by the survival rate.

Generating a token costs about 2 FLOPs per active parameter, and training on one about 6 [48]. Cost projections require reproducible inputs and calculations (Section 5.6).

For an observed batch, cost per verified win is the total ledger divided by verified wins, including update and operating costs where applicable. This accounting does not require independent attempts. Adaptive search still needs empirical cost and attempt curves; a batch with no verified win has no finite observed cost per win.

Training and deployment need separate ledgers. Training pays for every attempt, including failures. The proposed deployment protocol pays for encoding, one proposal and its fit, with no repair. Fixed-weight agent loops pay for evaluation during task-specific search. Training a proposer may shift some of that cost to earlier tasks; the crossover still needs measurement (Section 7).

5.2 Wasted moves

The ledger distinguishes stillborn attempts from valid losses. A stillborn attempt fails to produce a usable program under the declared protocol: generation is incomplete, parsing or type-checking fails, execution exceeds its limits or crashes, or outputs fail predeclared validity checks. An also-ran is valid but does not beat the incumbent.

The pilot funnel planned for Section 7 (Figure 11, panel a) traces those stages and the compute spent before each failed attempt stops.

If 90% of attempts fail to improve the incumbent, the independent-try model requires ten attempts per improvement in expectation. A typed language can exclude invalid structures before fitting (Section 4). Runtime, resource and output failures may consume part or all of the fit, and their incurred cost must be counted. The pilot must measure whether also-rans dominate. To isolate the writer's contribution, compare it under a common search budget and incumbent; changes to either can alter the win rate.

5.3 The correction tax and the reasoning dial

Here and in Section 5.6, we consider language-model writers for regression and the agent comparators of Sections 7.3 and 7.4. The compact reference network and its generation limit are specified in Section 6; the scaling examples here concern language-model writers.

Small models are cheap per token and can be dear per win. For generation alone, compute per token falls about 190-fold from a hypothetical dense 1.5T writer to an 8B one. With equal token counts and compute unit costs, the smaller writer can tolerate a win rate about 190 times lower before that component's cost per win matches.

If it needs three times the tokens per attempt, the generation-only ratio falls to about 60. Repair turns also re-read a growing context. Encoding and referee costs change the full-cost ratio; a threshold for the complete bill in Equation (2) requires counting every component.

Reasoning is the second dial. Reasoning tokens may cut stillborn moves, but each is billed as output. The planned pilot (Figure 11, panel b) compares cost per win over writer size and reasoning effort. A U-shaped curve is the hypothesis; the measurement decides where, and whether, the minimum sits.

5.4 Rewarding useful hallucination

A familiar solution can be a useful prior. Matching the incumbent's score earns a tie; promotion requires improvement under the same protocol. A *useful hallucination* here is an unfamiliar proposal whose value the referee verifies (Box 3). Possible exploration controls include temperature and entropy schedules, a curriculum past known solutions, and quality-diversity archives [62, 72]. Any novelty bonus must preserve the reward ordering in Section 4. These are design options.

The reference training procedure combines policy draws, grammar-uniform fill and random exploration to seed verified search (Section 6.5). An exploration study must declare those controls and their budget shares, including unsuccessful attempts; Section 7 reserves the quality–cost comparison.

Box 3: What a referee verifies

An unfamiliar program is a conjecture; the referee checks whether it runs and improves the declared score. Verification establishes that bounded result. A claim of discovery also needs comparison with known methods (Section 8). The Go example in Section 2.4 illustrates this distinction. The referee's reliability and execution cost must be established for the task (Section 6.6).

5.5 How many games?

Keep three quantities separate. **Tries to find one improvement:** $1/q$ on average (Equation (1)). For independent tries with fixed $q = 1\%$, 69 attempts reach even odds and 299 reach a 95% probability of at least one win in that model. Report empirical attempt curves for adaptive or correlated proposals.

Experience to train a transferable player: AlphaGo Zero generated 4.9 million self-play games in about three

days [101], and AlphaZero played 44 million games of chess in about nine hours [102]. An illustrative grid of 10,000 problems, 100 rounds and 8 attempts per problem per round gives 8 million attempts, the same order. The grid counts work, not learning: the number of problem families matters more, and pilot learning curves, read only within their measured range, must decide.

Precision to plan the test: illustratively, 150 independent units at a win rate near 60% give an approximate 95% interval of ± 8 points. The actual study must compute uncertainty over independent families, rather than treating every dataset as an independent observation.

5.6 Pricing evidence still required

Turning the per-token scaling comparisons of Section 5.3 into a hardware bill requires accounting for resident model memory alongside active compute. Mixture-of-experts writers use compute on active parameters but need memory for all parameters: Kimi K2 has 32B active of 1T, while DeepSeek-V3 has 37B active of 671B [27, 51]. Resident one-byte weights for 1T parameters alone require at least 13 H100 SXM GPUs with 80 GB each [76]; runtime state needs additional memory. This memory floor is separate from the cost estimates omitted below.

A reproducible projection therefore needs a pinned model and hardware configuration, dated prices, measured or justified utilization and an explicit accounting boundary. Those inputs remain unrecorded; a full cost projection remains a measurement task.

Key idea

Cost per verified improvement depends on both the cost of an attempt and the probability that it wins.

A fixed attempt budget becomes a cost per win only after measuring the win rate under a common task mix, incumbent and verification rule. Under Equation (2)'s fixed-probability model, doubling q halves expected cost per win only if mean attempt cost stays fixed.

Section 6 separates the offline writer-and-map reference from the later adapter admission loop. Their measured costs require separate ledgers: adaptive admission rates do not supply the writer's first-proposal win rate. That comparison remains the single-candidate test in Section 7.2.

6 EurekaZero: Learning Taste

The method has two distinct development stages. First, the offline multi-dataset reference trains a neural program writer and a separate ranking map from search-scored lists. The architecture and training rule below describe that reference; proposed deployment asks its writer for one new program. Second, Section 6.8 examines a closed-loop admission ratchet whose inspected implementation uses heuristic edits, retrieval and grammar proposals with adapter-only updates. Its retained logs do not qualify the reference writer or establish transfer between the stages.

6.1 The player in one picture

Figure 4 connects the architecture to the contract in Box 1: a frozen encoder reads the board and π_θ writes a complete program, as AlphaGo's policy proposed a move [100]. The problem–program map v_ψ supplies the seed ranking during training, without conditioning or screening the writer. Proposed game day omits that map and permits only the writer's single draw.

6.2 Reading the board

The reference regression encoder turns the board of Section 4 into 2,048 numbers, $z_p = \phi_{\mathrm{P}}(p)$, in three blocks. The first summarizes a pretrained row encoder's embeddings across rows, independent of row order (512 values). The second projects the hidden state of a pretrained third-party tabular regressor that reads the training rows and labels (1,024; third-party provenance pending; restricted; redistribution prohibited). The third summarizes two 32-element windows of training targets scaled to $[0,1]$ (512 values; constant targets map to zero).

Preparation caps the combined tabular splits at 50,000 rows. Encoder inputs remain training-side only: the regressor block samples at most 768 context rows and 256 query rows from those training rows. Frozen meta-training cohort moments standardize each component block before concatenation and elementwise clipping to $[-32, 32]$.

The reference encoder is pinned by hash and stays fixed during writer training. Encoding is bounded at 30 seconds per problem. A context with any non-finite value is refused rather than imputed (Appendix C). The input contract permits training-side task evidence only; final held-out labels and their derived statistics must stay outside the input closure. A context's recorded declaration that no final target was used establishes neither input closure nor numerical qualification. CPU contexts require native replay and bounded-execution receipts before accepted-policy use.

Whether problems with similar best programs sit together requires a separate representation study; Section 7.5 reserves that test.

6.3 Describing the move

A move is one program in the typed language of Section 4, with a 401-token training inventory listing 31 transforms and 22 prediction heads; the product runtime restricts executable heads. The writer emits a flat operator-and-slot token stream with a fixed beginning, engine and regression-task header, rather than parenthesis tokens or the `pipe` wrapper. Its nine-phase grammar mask permits up to six transform stages and three expression stages; each expression tree is bounded at depth four and 15 nodes [24, 56]. After the end token, detokenization renders a parenthesized program that must parse before entering the pool. The mask has no remaining-token-budget check: a draw without an end token at the 128-token

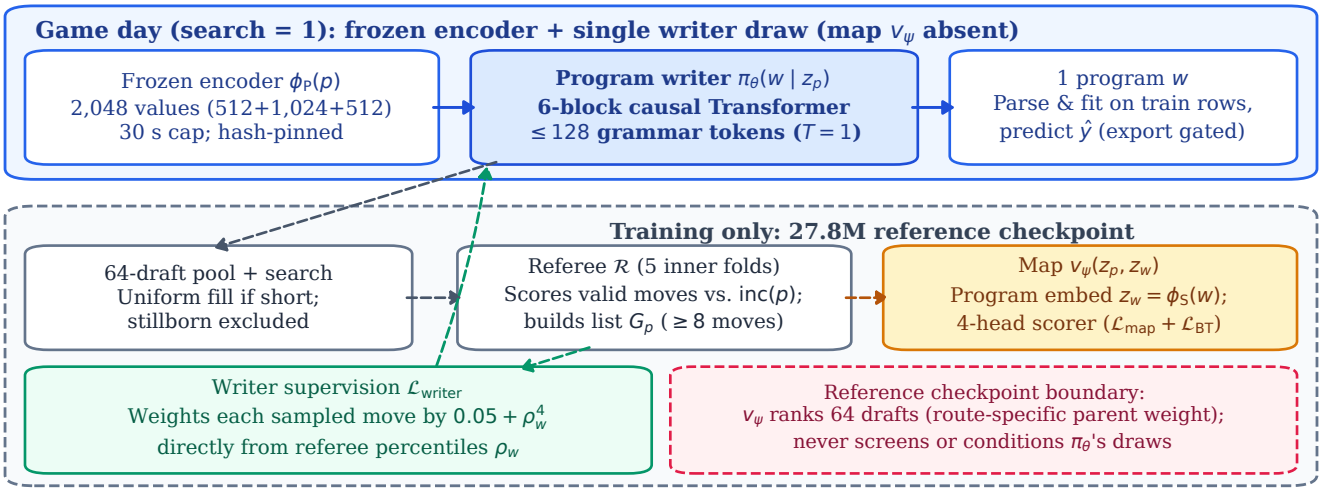


Figure 4: **A writer and a separate ranking map learn from scored programs.** Historical reference schematic. Top: game day, the whole of search = 1. Dashed box: training only. Search seeded with the writer's drafts proposes more programs, the referee scores them against the incumbent, and the scored lists retrain the writer (θ) and the map (ψ). Stillborn programs enter no list.

horizon, including the fixed header, is incomplete and discarded. Grammar legality does not establish successful execution or useful quality. The proposal pool removes exact decoded-string duplicates and records their SHA-256. The executor also records canonical recipe hashes. Probe-based behavior matching is a separate audit, not part of pool deduplication. Without probe evidence, this behavior check reports novelty as unknown. Readable Python and Rust exports are a reference requirement. Legacy recipe emitters reject the synthesized input stage; the separate fitted-state path is allowlisted. Reported winners still require the parity check in Section 6.6.

6.4 One map for problems and programs

The map reads a program as $z_w = \phi_S(w)$, the layer-normalized mean of non-padding token embeddings, including `BOS`, the fixed engine and task header, and `EOS`. Pooling discards token order, limiting its ability to distinguish compositions built from the same tokens.

The map's own two-layer context network projects the 2,048-value z_p to $r_z \in \mathbb{R}^{512}$. It joins three 512-value blocks: z_w as is, $z_w \odot \tanh(W_\gamma r_z + b_\gamma)$, and $|z_w - (W_{proj} r_z + b_{proj})|$. Four two-layer heads receive this same 1,536-value interaction vector; their mean gives the ranking logit $v_\psi(z_p, z_w)$. Native ranking losses supervise that mean jointly, without separate targets or bootstrap samples. Training applies dropout of 0.05 in each ranking head. The runtime head instead reads $[r_z, z_w]$, while the program-only control reads z_w.

Higher logits rank higher within a problem. The score is not a calibrated error estimate. Comparing its magnitude across problems or reading it as a probability of a verified win requires separate evidence. The referee still establishes validity and measures performance.

The map learns from scored lists G_p for problem p; $\rho_w \in [0,1]$ is a program's percentile, 1 for the best. Native training requires at least eight scored programs in a source list and samples eight per comparison: its champion and seven quantile-stratified comparators. The lower-level listwise loss accepts groups of at least three with finite percentile targets (Appendix C). The list-level form trains the map's softmax toward a target concentrated on the best [16]:

$$\mathcal{L}_{map}(p) = -\sum_{w \in G_p} \frac{e^{\rho_w/\tau}}{\sum_{u \in G_p} e^{\rho_u/\tau}} \log \frac{e^{v_\psi(z_p, z_w)}}{\sum_{u \in G_p} e^{v_\psi(z_p, z_u)}}, \quad (3)$$

with $\tau = 0.25$. With a unique best program, the low-temperature limit concentrates the target on that program. This is a listwise ranking objective over programs scored on the same problem; the analogy to contrastive learning [88, 111] does not establish CLIP-style alignment across problems and programs. A Bradley–Terry term adds pairwise comparisons on the same lists [12]. Stillborn moves have no score, so this ranking objective covers only programs that run.

A program-only head trained beside it provides a control for testing whether the map uses problem context (Section 7).

Key idea

The reference game-day protocol omits the ranking map. Its single proposal tests what the writer has learned.

In the native implementation, the writer and map use separate context networks and token embeddings. The map ranks the draft pool without dropping drafts or screening the writer.

The map can influence search through parent selection. One native GP route weights seeded-survivor parents by map rank; the bounded search driver instead cycles among up to four parents chosen by measured loss. Any claimed effect on training must identify which route was used.

Only the 64 neural or fallback seeds receive normalized rank weights; the 16 tail seeds enter unscored. Unscored survivors share the lowest scored survivor weight while scored survivors remain.

Native assembly takes up to ten same-cut teachers, then cross-cut teachers up to a 16-seed tail. Random fallbacks fill to ten slots if needed; random exploration fills the remaining slots. The standalone seed writer instead uses 16 random programs. Teacher-first execution prioritizes retained cut teachers; Algorithm 1 leaves the tail route-dependent.

This search role is separate from the writer loss, which weights programs by referee rank. The program-only control tests context-sensitive ranking; a change in proposal quality requires the matched ablation in Section 7.5.

6.5 The self-play loop

The native proposal sampler seeks 64 distinct parseable programs at $T = 1$, scheduling at most eight batches of 128 neural draws (1,024 total). To fill a shortfall, it tries up to 10,000 seeded random canonical pipelines and projects them through the grammar. It refuses an incomplete pool. These are training-search budgets, separate from the one-draw protocol in Section 6.7.

The reference training rule is expert iteration [1]: search-scored programs train the writer. The writer term weights sampled programs from G_p by referee rank; it joins the map and auxiliary terms in the combined objective (Appendix C):

$$\mathscr{L}_{\text{writer}}(p) = -\sum_{w \in G_p} \left(0.05 + \rho_w^4\right) \log \pi_\theta(w \mid z_p), \qquad (4)$$

$$\mathscr{L} = \mathscr{L}_{\text{writer}} + 0.1\,\mathscr{L}_{\text{map}} + 0.25\,\mathscr{L}_{\text{BT}} + 0.1\,\mathscr{L}_{\text{runtime}} + 0.05\,\mathscr{L}_{\text{prog}}. \qquad (5)$$

The writer loss in Equation (4) is schematic: it displays the rank weights. The native implementation also smooths grammar-legal targets, averages token losses within each program, and normalizes by the sum of program weights (Appendix C).

Here $\mathscr{L}_{\text{BT}}$ is a Bradley–Terry loss on the same lists, $\mathscr{L}_{\text{runtime}}$ a Huber loss between $\ln(1+s)$ and $\ln(1+\text{softplus}(h))$, where s is nonnegative measured seconds and h is the raw runtime-head output, and $\mathscr{L}_{\text{prog}}$ the Bradley–Terry loss of the program-only control head. A median-ranked program carries about a tenth of the top-ranked program's weight in the writer loss. Its referee score also contributes to the map's ranking targets.

The reference procedure uses two ratchets (Section 4). A program displaces a board's incumbent only by strictly beating it.

Under the reference admission rule, a retrained writer faces 16 cuts with three seeds each. It replaces the serving writer only if its mean log error ratio favors it. It must also win at least as many cuts as it loses and fail no more often. Here a cut is a held-out evaluation case fixed before the admission test. For each cut, the rule takes the natural log of paired best-error ratios (challenger divided by serving) and averages over matched seeds; it then averages across cuts. Negative values favor the challenger. This admission comparison is separate from the single-program comparison in Section 7.2.

The historical reference. The writer is a decoder-only transformer (six blocks, width 512, four attention heads). It reads the context as a prefix token and writes at most 128 tokens over 401 active grammar tokens. Its token table allocates 657 rows, including 256 unused reserved rows; the tied output projection uses only the grammar rows. With the map and registered auxiliary heads, the checkpoint contains 27,796,494 parameters (27.8 million). Appendix C distinguishes registered capacity from the token-generation path. The supplied source notes describe training on eight A100 GPUs in September 2026, with a corpus of about 10.2 million scored programs from 80 training datasets. Algorithm 1 and Appendix C give the reference specifications.

These specifications do not supply the checkpoint's acceptance evidence or qualify its runtime. Both records remain inputs to the evaluation contract (Section 7.1). The accepted native checkpoint route is limited to tabular regression, and the native training entry point remains held at its grant gate. Numerical parity of the CPU float32 Candle port with the native CUDA BF16 tch runtime remains unqualified; random streams and uniform-token fallback differ.

6.6 Keeping the referee honest

Search supplies training targets only as reliably as its referee. A learner rewarded by a referee can exploit its flaws. In February 2025 Sakana AI walked back claims after its AI CUDA Engineer exploited the evaluation code [116]. In June 2025 METR reported frontier models patching their scorers [114], and 16% of 1,968 agent-benchmark tasks can be passed without being solved [123].

The evaluation protocol must withstand such attacks. Selection, promotion and final scoring must use separate data (inner folds, held-out cuts, a final fold scored once), with sealed meta-test families outside the learner's inputs.

The proposed comparisons require sandboxed execution using seccomp and Landlock, with memory, swap and time limits. A planted label leak and a sealed canary must both be caught. Every reported winner must be re-run from its emitted code, with native and emitted predictions agreeing within $10^{-9}\max(1, |\hat{y}|)$.

One earlier leaderboard entry for our search selected each dataset's best test-scored outcome from a ledger of 63,108 evaluations; we exclude it as test selection (Appendix B).

6.7 Game day: search = 1

Algorithm 2 instantiates the contract in Box 1 for regression, with a recorded seed and frozen components. That contract fixes the candidate budget, failure handling and full cost boundary.

Report later search separately, as Silver et al. [101] reported AlphaGo Zero's raw network apart from its search (Section 7).

That fixed budget defines the matched first-proposal comparison in Section 7.2. The development record that follows concerns an adapter ranking an existing pool. Its admission decisions measure a different task from writing a new program.

Algorithm 1 The reference self-play loop (training).

Require: training problems; serving writer π_θ and map v_ψ; incumbents $\mathrm{inc}(p)$

for each round **do**
  for each sampled training problem p **do**
    $z_p \leftarrow \phi_{\mathrm{P}}(p)$; keep the first 64 distinct valid draws from $\pi_\theta(\cdot \mid z_p)$ ▷ $T = 1$; bounded fallback if short
    verified search, seeded with the 64 drafts and 16 tail seeds (teachers/fallbacks and random exploration), proposes more programs ▷ the teacher
    $\mathscr{R}$ fits and scores each distinct program on 5 inner folds ▷ stillborn: no score
    if a program strictly beats $\mathrm{inc}(p)$ **then** it becomes $\mathrm{inc}(p)$ ▷ board ratchet
    add every scored program to the list G_p ▷ winners and also-rans
  train a challenger (θ', ψ') on the lists
  if it beats π_θ on held-out cuts **then** $(\theta, \psi) \leftarrow (\theta', \psi')$ ▷ player ratchet

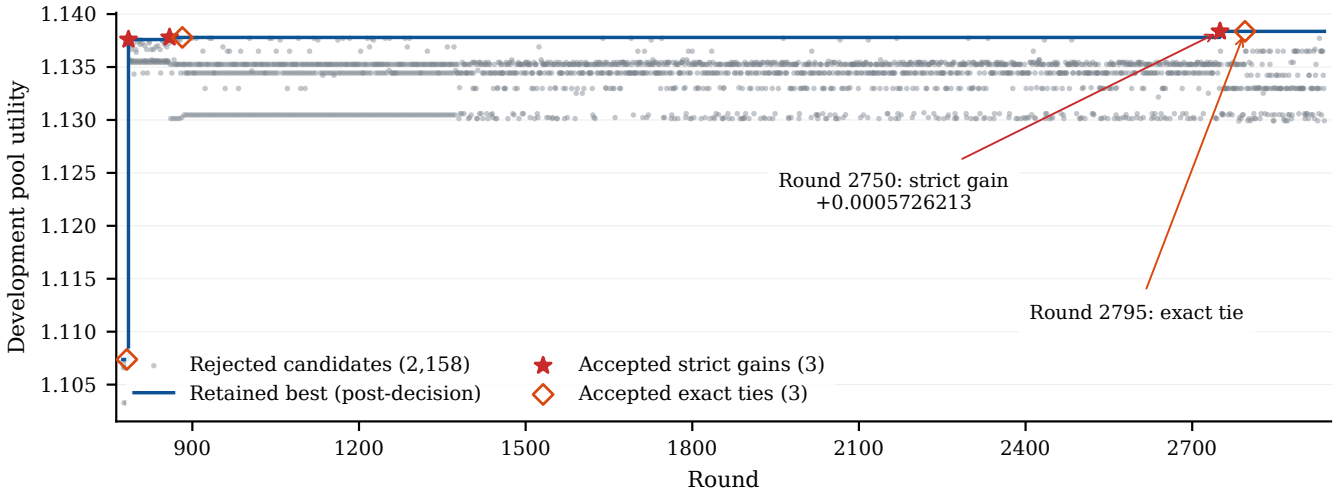


Figure 5: **Retained development utility through round 2938.** All 2,164 admission records from rounds 775–2938 are shown: blue is the post-decision retained best; grey marks rejected candidates, red stars accepted strict gains and hollow diamonds accepted exact ties. The default rule accepts candidate utility $\geq$ the previous best, without a margin. Acceptance can therefore change the recorded best-round identity without raising utility. Ties are classified from unrounded logged values. These adaptive development observations require separate first-proposal and transfer evaluations.

Algorithm 2 Proposed game-day protocol (search = 1).

Require: new regression problem p with training rows and metric; frozen ϕ_{P} and π_θ

$z_p \leftarrow \phi_{\mathrm{P}}(p)$ ▷ read the board
$w \sim \pi_\theta(\cdot \mid z_p)$, one draw at $T = 1$ ▷ one move; no retry
$f \leftarrow \textsc{Fit}(w,$ training rows$)$ ▷ as any algorithm is fit
$\hat{y} \leftarrow f($new rows$)$ ▷ a stillborn w scores as a loss
return w, $\hat{y}$

6.8 A separate adapter admission record

The retained CSVs summarize a later regression admission record (Figure 5), separate from the historical reference above. An adapter is the trainable adjustment evaluated by this admission loop. Utility is the logged scalar score, with larger values treated as better. The admission CSV records rounds, candidate and best validation utilities, acceptance flags and timestamps. The tables omit the adapter architecture, score normalization and evaluation population, so they do not support comparisons with other models. The source inspection that follows describes the loop implementation. It does not yet bind that source or the adapter's identity to the recorded run. Logged utilities alone do not identify the snapshot's relationship to the historical writer or map.

In the inspected loop implementation, heuristic program edits, retrieval and grammar sampling supply candidate programs. Only an adapter that ranks proposals by predicted cost is optimized; the encoder and other modules stay frozen. This loop has no neural program writer. Its pool-utility calculation uses the recorded scores and gate outcomes of the first four candidates as revealed history, then selects the candidate with the lowest predicted cost from the pool, including those entries. This is a conditional ranking task, separate from a single new program drawn by the historical writer. Training puts the lowest recorded scores first in its slates; utility evaluation uses stored pool order. The two calls can therefore reveal different examples.

The inspected caller supplies modality tokens and a mask while zeroing separate context and prior inputs. Its objective combines regression to recorded costs and listwise matching, with a fixed target for failed gates. The reporting code averages

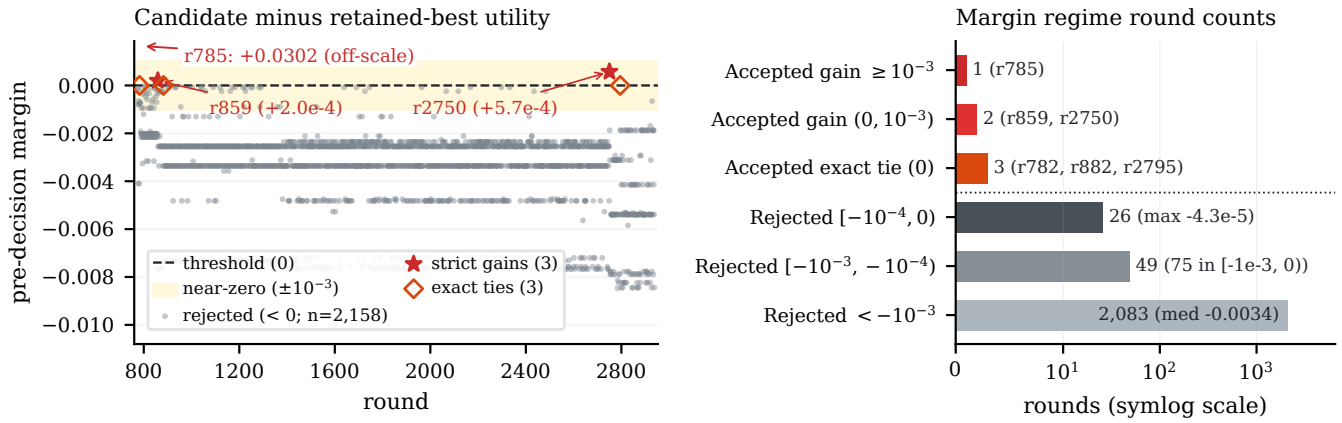


Figure 6: **Pre-decision validation margins across rounds 775–2938.** Left: candidate minus prior retained-best validation utility ($\Delta_r = u_{\text{cand},r} - u_{\text{prev},r}$) across all 2,164 retained admission records in rounds 775–2938, zoomed around the $\Delta_r = 0$ admission threshold (round 785's +0.0302 gain is annotated off-scale). Right: round counts by pre-decision margin band on a symmetric log scale. Half of all six accepted rounds are exact ties ($\Delta_r = 0$ at rounds 782, 882 and 2795), two are sub-10^{-3} gains ($+1.99 \times 10^{-4}$ at round 859 and $+5.73 \times 10^{-4}$ at round 2750), and 26 of the 2,158 rejected candidates lie within $[-10^{-4}, 0)$ of a tie. This is development telemetry from the same retained window as Figure 5, not a Results claim or an independent first-proposal evaluation.

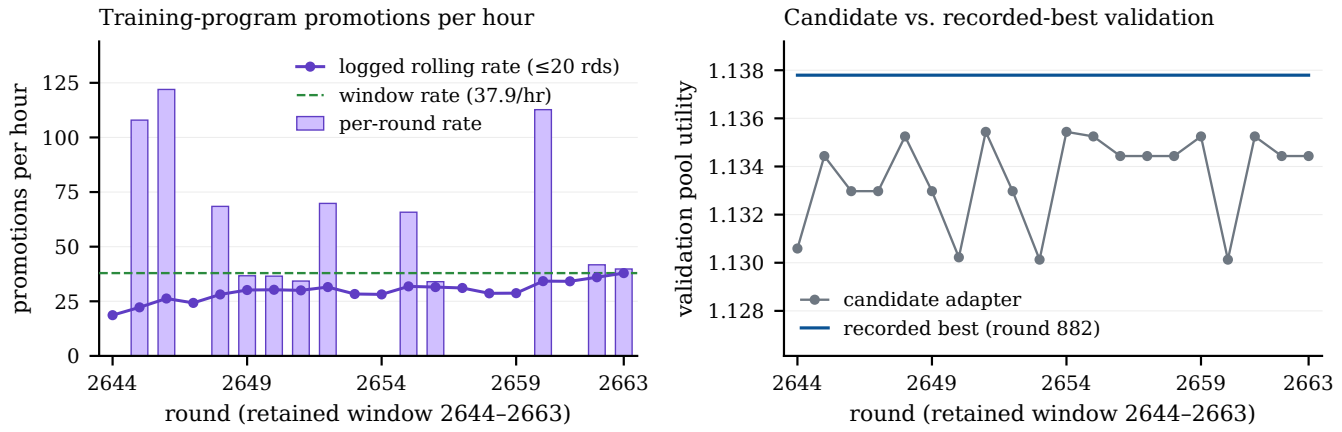


Figure 7: **Training-program promotions per hour and adapter validation trajectory.** Left: logged rolling and per-round strict training-program promotions per hour. Right: candidate and recorded-best adapter validation pool utility. Both panels cover the 20 consecutive completed rounds 2644–2663 (21 strict promotions over 1,994.54 s of logged wall time, or 37.90 promotions per hour, with zero adapter admissions).

the negative selected scores, assigning a fixed penalty to failed gates.

Candidate programs are evaluated on two training-side folds; the historical reference procedure specifies five. Evaluations are logged separately from training-pool admission: ordinary admission requires passing gates and a recorded score below the training context's incumbent by more than 10^{-6}. Historical programs have a separate admission path. The adapter's default gate can accept equal utility; rejection restores model and optimizer states.

[TBD: Bind the retained snapshot to the exact loop source, adapter, candidate pools, revealed-history inputs and utility normalization.]

One retained admission ledger. Figure 5 covers all 2,164 captured rounds, 775–2938: six admissions and 2,158 rejections. Strict gains occur at rounds 785, 859 and 2750; exact ties occur at 782, 882 and 2795. The first 1,889 rows end at round 2663. The final 275 add the gain at 2750 and tie at 2795. Round 2750 raises retained utility from 1.1378 to 1.1384 (rounded), an increase of about $+5.73 \times 10^{-4}$. A change in recorded best-round identity alone is not a utility gain. Figure 6 separates those outcomes using candidate minus prior retained-best utility; Appendix D.9 gives the margin distribution for this same ledger. The 101 records added after round 2837 are all rejections; the retained-best utility is unchanged through round 2938.

Program-incumbent events count improvements to training programs, whereas adapter admissions concern the ranking adjustment. Figure 7 puts the former on a time scale for rounds 2644–2663; its paired utility panel records no adapter

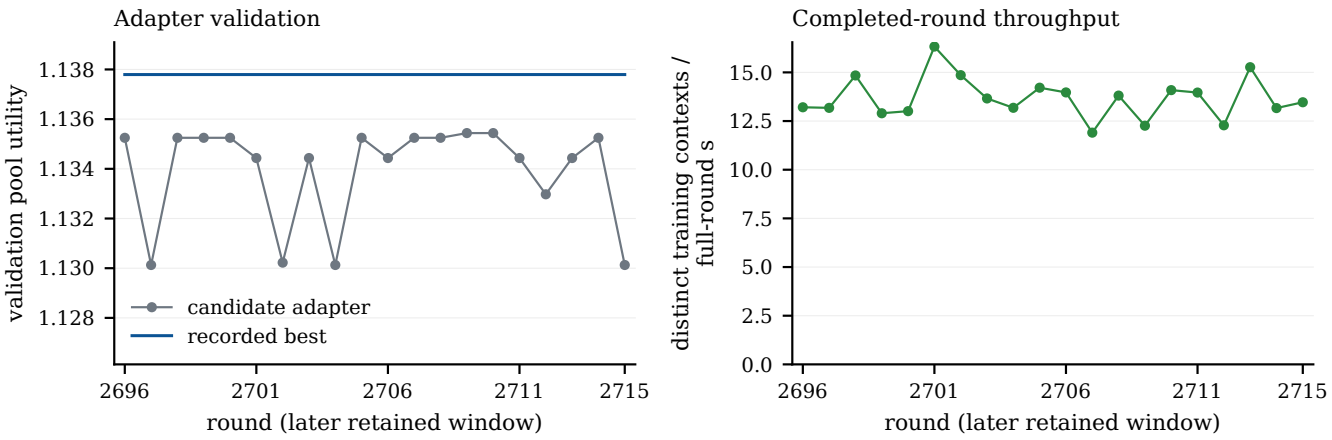


Figure 8: **Adapter utility and whole-round throughput in a later retained window.** Both panels cover the same 20 completed summaries, rounds 2696–2715. Left: candidate adapter validation utility and the recorded best. Right: distinct training contexts divided by the full wall time of each completed round. The rates include all work and waits inside that interval. These are development operating observations; independent first-proposal and deployment comparisons remain required.

admission. These outcomes need separate denominators and cannot stand in for one another.

Figure 8 measures distinct training contexts divided by full completed-round wall time in rounds 2696–2715. It includes work and waits inside each round; it is neither deployment throughput nor a count of distinct tasks across rounds. The operating windows differ from the full admission prefix, so these panels do not form a matched learning curve.

The remaining windows, alternative summaries and historical GP comparisons are collected in Appendices D and D.10. They document the retained record without adding independent evaluations. Section 7 specifies the matched tests needed to establish whether training improves the writer's first proposal on new families.

7 Results

The independent tests below ask whether training improves the first proposal for an unseen problem. The development records in Section 6.8 track program search and adapter admission. Those decisions do not establish first-proposal performance at deployment. Table 5 summarizes the pending comparisons, controls and status. **[TBD: Add independent evaluation evidence before making a performance claim.] [TBD: Supply a source-bound benchmark before claiming deployment throughput.]**

7.1 Evaluation contract

Freeze the writer, permitted task inputs, output support, decoding rule and budgets before opening final outcomes. Pin dataset versions and content hashes, row splits and family assignments. Separate unseen problem families from training and development, and report which task types were actually tested.

Report known training and pretraining exposure for the writer, encoders and learned comparators, with unresolved provenance marked explicitly. Bound unseen-family claims to the exclusions actually established.

Each EurekaZero submission follows Algorithm 2; failed generations and executions remain in the denominator. Report paired results and uncertainty at the family level, alongside encoding, generation, fitting and prediction costs. Disclose internal model or hyperparameter selection within the emitted procedure, even when the writer submits only one outer candidate. **[TBD: Document the final problem inventory, dataset identities, row splits, family assignments and exclusions, component provenance, evaluator, checkpoint, seeds, task metrics and failure accounting.]**

7.2 The effect of training

Use a frozen reference writer as the control. Predeclare its checkpoint and the evaluated training intervention before opening final outcomes. Hold the encoder, program grammar and output support, permitted task inputs, decoding rules, fitting protocol, referee and resource budgets constant; vary the writer checkpoint. Changes to the support, encoder or evaluator belong in separate conditions.

Pair the first draws under predeclared seeds and report all of them, with no selection of the best seed or candidate. Report paired family-level outcomes, failures and costs alongside source-bound evidence of optimizer updates and parameter changes. **[TBD: Supply the matched first-proposal comparison and training-update evidence before claiming transfer.]**

7.3 One proposal against fixed methods

[TBD: Identify Jepson's supplied frozen comparators, their provenance and eligibility, and the matched outcomes and uncertainty for Table 6 and Figure 9.]

Table 5: **Evaluation plan at a glance.** Each test is specified in the subsection named in the first column. No outcome is reported yet; the contract in Section 7.1 fixes inputs, budgets and splits before outcomes are opened.

Test	Question	Comparison or control	Reported in	Status
The effect of training (Section 7.2)	Does training improve the first proposal on unseen families?	A frozen reference writer under the same budget, paired first draws	Text of Section 7.2	Pending
One proposal against fixed methods (Section 7.3)	Does one task-conditioned program compete with methods that run unchanged across tasks?	Frozen algorithms whose receipts record the designing model	Table 6 and Figure 9	Pending
The cost of more attempts (Section 7.4)	What do successive attempts buy, per attempt and per unit of cost?	Agents allowed successive attempts; the pilot cost funnel	Figures 10 and 11	Pending
What the map contributes (Section 7.5)	Does the map read the problem, and does it change the writer?	The program-only head; a matched causal ablation	Table 7	Pending
Failure cases and a worked example (Section 7.6)	How do attempts fail, and what does one game look like?	Terminal outcome classes; a rule-selected example	Table 8	Pending

Table 6: **Planned paired comparisons.** No outcomes are yet supplied. Report ties, failures and excluded cases alongside wins.

Comparator	Outcome and uncertainty
[TBD: first frozen comparator identity]	**[TBD: paired evaluation]**
[TBD: second frozen comparator identity]	**[TBD: paired evaluation]**

[TBD: Leaderboard: supply the evaluated methods, paired outcomes, rating protocol and uncertainty.]

Figure 9: **Leaderboard pending.** Elo is a summary of a specified comparison population, not a substitute for paired task results.

[TBD: Search curve: performance versus attempts and compute, including failed attempts and unreached targets.]

Figure 10: **Deployment search comparison pending.** The single-candidate condition fixes a budget; its measured quality remains to be established.

7.4 The cost of more attempts

Compare the single proposal with agents allowed successive attempts and development feedback. Keep final outcomes unavailable to those agents. Count every candidate attempt, including incomplete or invalid outputs and failed executions; a repair that emits a revised candidate is another attempt. Charge all task-time computation.

Plot performance against both attempts and cost (Figure 10); if an agent does not reach a reference performance within its budget, report that limit. **[TBD: Supply attempt curves, matched budgets, repeated seeds and uncertainty.]** Pilot cost measurements from Sections 5.2 and 5.3 are planned for Figure 11.

[TBD: Pilot cost measurements: supply per-stage failure shares and cost per verified win.]

Figure 11: **Pilot cost measurements pending.** Planned panels: (a) the share of development proposals that run, score, beat the incumbent and survive re-verification, with the compute spent at each stage; (b) cost per verified win by writer size and reasoning effort. **[TBD: Supply pilot data, seeds and family-level uncertainty; identify settings with no verified win without assigning a finite cost per win.]**

Table 7: **Map evidence still required.** Ranking quality and a change in first-proposal quality are separate outcomes.

Question	Required evidence
Does the map rank candidates?	**[TBD: held-out ranking study]**
Does it improve the writer?	**[TBD: matched causal ablation]**
Does it reduce execution failures?	**[TBD: failure-class comparison]**

7.5 What the map contributes

Test whether nearby problem encodings share useful programs, using data disjoint from the data that fit the map. Compare ranking agreement with the program-only head, so that a global preference for common programs can be distinguished from reading the problem. **[TBD: Supply problem-similarity and program-only-control evaluations.]**

A ranking head can agree with a referee without improving the writer (Table 7). In the checkpoint described in Section 6.4, the map neither conditions nor screens the writer's draw. Its ranking accuracy therefore cannot establish a reduction in wasted proposals.

In one native GP route, the map's ranks weight parent selection. A causal ablation must bind this route to the evaluated checkpoint and vary the map's influence with the writer, training budget and deployment protocol otherwise matched.

Table 8: **Worked-game record.** Fields reported for the game selected by the rule above. An empty selected outcome category is reported rather than replaced.

Field	Record
Problem, task and metric	**[TBD: Identify the problem, task and metric.]**
Encoding time	**[TBD: time]**
Emitted output	**[TBD: program or unfinished draft and size]**
Check and fitting outcomes	**[TBD: outcomes and time]**
Hidden-fold score of a valid fit	**[TBD: score]**
Incumbent's score	**[TBD: score]**
If no valid fit	The failure and its incurred cost, in place of the scores

[TBD: Identify an implemented influence path or limit the study to ranking.]

7.6 Failure cases and a worked example

Describe unsuccessful proposals and coverage limits as well as wins. Assign each attempt one terminal status: incomplete generation, parse or type-checking refusal, execution or resource failure, invalid output under declared checks, and valid ties, losses and wins. Charge every attempted candidate its incurred cost. Unresolved outcomes retain an explicit unknown status; fix their treatment and the validity rules before opening final outcomes. **[TBD: Supply attempt inventories and costs by outcome class, identifying unresolved cases.]**

Select the worked example by a rule fixed before inspection. Report its emitted program, fitting procedure and comparison outcome. If the selected category is empty, report that fact. **[TBD: Supply the example selection rule, actual program and verified outcomes, with no replacement of failed attempts.]** Table 8 lists the record for the selected game. The attempt permits no repair or second try, and its sealed verdict is reported without updating the writer.

7.7 Earlier development diagnostics

[TBD: Supply Jepson's source-bound search and ranking diagnostics, including original inventories, comparison conditions, overlap checks and uncertainty. Keep their scope separate from the final first-proposal comparisons.]

[TBD: Pin the inventories, checkpoints and seeding, timing and ranking protocols, separately from the final comparisons above.]

[TBD: Supply the interpretation and limits of these comparisons in Section 8 once Jepson provides the source.]

8 Discussion

[TBD: Supply a source-grounded discussion of the completed evaluation from Jepson. Reconcile every conclusion below with those outcomes and their uncertainty.]

8.1 Learning to select and learning to invent

[TBD: Use Jepson's supplied output and archive comparisons, with program behavior checks, to distinguish selection from useful new composition.]

8.2 Moving judgment across problems

[TBD: Interpret transfer of first-proposal quality across problems using the matched conditions and paired family-level uncertainty supplied by Jepson.]

8.3 The referee defines the boundary

[TBD: Discuss referee validity, contamination, admission versus generalization, and documented failure limits from Jepson's source.]

8.4 A bounded claim about self-improvement

[TBD: State which level of self-improvement is supported by source-bound training changes and independent proposal outcomes supplied by Jepson.]

8.5 Limitations

[TBD: Describe limits on task coverage, the program language, referee reliability and measured cost using Jepson's supplied source.]

9 Conclusion

[TBD: Supply the conclusion and next steps justified by Jepson's results and their limits; reserve stronger claims until their required evidence is supplied.]

A Referee Inventory

Section 3 traces the four pillars of Section 2 through the milestones used here: algorithm selection [90], typed genetic programming and evolutionary search [53, 71, 89], self-play and expert iteration [1, 93, 101], and task-conditioned amortization across tabular problems [34, 44, 73]. Each of those systems depends on a referee. Table 9 collects the main text's referees by domain. A domain enters the innovation game only when its referee is reliable and affordable (Section 4.5).

B Incident Log and Audit Safeguards

The reliability requirements in Section 6.6 distinguish a documented baseline exclusion from checks still required for final evaluation. Artifacts that violate held-out isolation belong in the audit record:

- **Excluded test-selected search baseline.** An earlier internal leaderboard entry selected each dataset's best test-scored outcome from 63,108 evaluations in total across 42 datasets. Selection using test-fold scores biases the reported metric upward [17]. The entry is therefore excluded from all comparative claims.

Table 9: **Referees by domain.** Each row restates a referee described in Sections 3 and 4; the last row states the reference referee for algorithm invention.

Domain	Referee	Systems
Board games	The rules of the game and its final score	AlphaGo Zero, AlphaZero [101, 102]
Machine-learning research	A sealed test set that scores every entry	Netflix Prize, ImageNet [7, 29, 92]
Code	Unit tests; a code executor	Codex, Absolute Zero [19, 122]
Matrix multiplication	The rules of TensorGame	AlphaTensor [33]
Sorting	Sorting tests	AlphaDev [66]
Mathematical constructions	Programmatic evaluators	FunSearch [91]
Accelerator kernels	Timers	AlphaEvolve [75]
Formal proofs	The Lean proof checker	AlphaProof, STP [28, 45]
RNA design	Laboratory tests of designs	EteRNA [60]
Algorithm invention (this paper)	Hidden-fold scores against the incumbent; promotion on strict improvement	EurekaZero (Sections 4.4 and 6)

- **Input-closure and canary checks.** Before benchmark release, the evaluation protocol requires passing receipts for an inner-fold label-poison test, a sealed canary check, and a train-only input-closure audit verifying that held-out test targets and their derived statistics do not enter board encoding or candidate generation. The final pre-registration and audit receipts belong to the evaluation record in Section 7.1; they are not reproduced here.

C Training Details

These implementation specifications describe the historical reference checkpoint in Section 6, separate from the later adapter snapshot in Section 6.8:

- **Board encoder blocks (Section 6.2).** The frozen 2,048-value context $z_p = \phi_{\mathrm{P}}(p)$ joins three blocks: (i) 512 pooled values from a pretrained row encoder of width 128 across training rows by mean, population standard deviation, and 10th and 90th percentiles; (ii) 1,024 values projected from a 2,048-value third-party tabular regressor state capped at 768 context rows and 256 query rows (third-party provenance pending; restricted; redistribution prohibited); and (iii) 512 values from two frozen seed-17 target-summary windows. Each block is normalized before concatenation and clipping to ± 32. Non-finite contexts are refused. Encoding records report wall-clock latency from 11.7 s to 20.8 s across eight reference encoding jobs, within the 30 s cap.
- **Comparison construction (Sections 6.4 and 6.5).** Lists G_p require at least eight valid scored programs per problem, with percentile $\rho_w = 1 - \mathrm{rank}/(|G_p| - 1)$ and writer sequence weight $0.05 + \rho_w^4$. The native trainer gives problem families equal mass and forms each training comparison from one list: its best program plus one program drawn from each of seven equal rank bands of the rest. Ranking pairs stay within these comparisons; reduction across them is described below. The Python ranking helpers group by dataset identifier. Native ranking groups by corpus context and the eight-row comparison index; separate comparisons from the same context do not pair with each other. Pairwise ranking omits non-finite targets and exactly equal values, $q_w = q_u$, leaving only strict finite orderings. Equations (3) and (4) show the list-level form.
- **Sequence supervision (Equation (4)).** The native decoder prepends the projected context token before `BOS`. The sequence loss leaves `BOS` unsupervised but includes the following non-padding tokens, including the engine and regression-task header and `EOS`. That header is fixed during inference. Label smoothing of 0.05 applies over legal tokens; token losses are averaged within each program before the normalized rank-weighted mean over programs. The displayed aggregate coefficients match the native objective; the checkpoint's objective identity and configuration still belong in its evaluation record.
- **Runtime supervision (Equation (5)).** The runtime term compares $\ln(1+s)$ with $\ln(1+\mathrm{softplus}(h))$ by Huber loss with threshold 1, for measured seconds $s \geq 0$ and raw head output h.
- **Ranking targets (Equations (3) and (5)).** In Equation (5), $q_w = 1/(1+\mathrm{RMSE}_w/m_p)$, where m_p is the corpus list's median RMSE. Bradley–Terry weights use $|q_w - q_u|$, doubled when either member is at or above the 90th percentile, then reduced over all non-tied within-comparison pairs in the rank-local microbatch. The implementation mean-scales their weights, casts them to float32 and averages weighted pair losses. Listwise losses are computed per eligible comparison group and averaged across groups.
- **Registered parameter inventory (Section 6.4).** The network has 27,796,494 parameters in total (six decoder blocks, width 512, four attention heads, prefix context token, and maximum length 128 tokens over the 401-token grammar). Each of the writer and ranking token tables allocates 657 rows: 401 grammar rows plus 256 reserved rows. The reserved rows are never indexed as valid input or output tokens; the tied writer projection is restricted to the grammar rows. The registered total also includes zero-weight stability and loss-scale terms, so it does not measure learned or exercised capacity. Approximately 6.90 million parameters lie outside the writer's token generation path. Decoding bypasses them, but the native checkpoint loader still requires them.

- **Optimization notes (Section 6.5).** The records describe AdamW training (learning rate 10^{-4}, weight decay 0.01, gradient-norm clip 1.0, global batch size 2,048) on eight A100 GPUs in September 2026 over 10,214,332 scored programs from 80 training datasets (7,877 lists; 8,894 contexts). These historical reference specifications are separate from the later adapter snapshot in Section 6.8. This paper does not document the evidence on which the checkpoint was accepted; that record and a held-out promotion receipt are required inputs to the evaluation contract (Section 7.1).
- **Execution sandbox and parity (Section 6.6).** The final evaluation protocol requires candidate isolation with seccomp and Landlock, cgroup memory limits with swap disabled, and native versus emitted-code agreement within $10^{-9}\max(1, |\hat{y}|)$. Isolation and parity receipts from the final evaluator belong to the same evaluation record.

D Flywheel Telemetry

The main text presents four views of admission and operating activity. This appendix supplies timing and event counts (Appendices D.1 and D.2), later operating windows (Appendices D.3 to D.6), and separate adapter-admission and program-promotion rates (Appendices D.7 and D.8). Appendix D.9 adds 275 captured rows through round 2938; earlier admission windows and historical GP comparisons appear in Appendix D.10. These views reuse retained records and add no independent evaluations. Binding the records to an exact loop, adapter and qualified runtime remains pending (Section 6.8); first-proposal comparisons belong to Section 7.

D.1 Development measurements over completed-round time

Figure 12 shows the same retained summaries as Figure 24. The horizontal axis sums completed-round wall time, including qualification waits. Each point is an end-of-round summary; the event count starts from zero for this window.

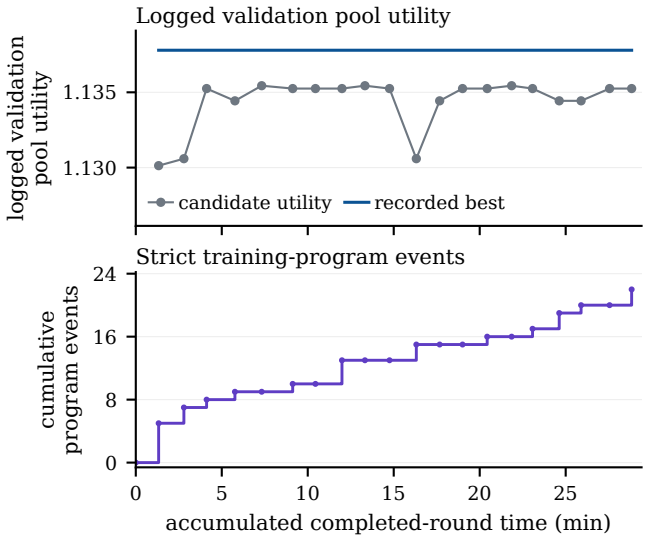


Figure 12: **Training-program events and logged validation utility over time.** Top: candidate and recorded-best validation pool utility. Bottom: cumulative strict training-program incumbent events reported by round end. Both panels use all 20 consecutive summaries from rounds 2343–2362. The summaries mark no adapter admission. These development observations do not establish independent first-proposal improvement.

D.2 Event counts in retained development windows

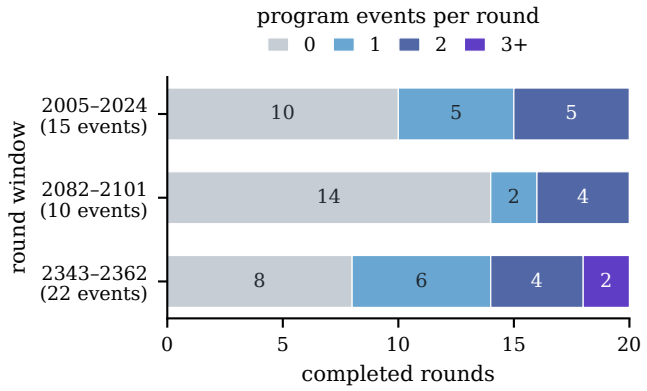


Figure 13: **Rounds grouped by training-program incumbent events.** Every bar contains 20 completed-round summaries. Colored segments count rounds with 0, 1, 2 or 3+ strict training-program incumbent events; parentheses report total events in each window. These descriptive counts are not a matched comparison or evidence of independent first-proposal improvement.

Figure 13 summarizes the training-program counts underlying Figures 21, 22 and 24. Each window stays separate; the records say nothing about the gaps between them. These are incumbent events within training contexts, not independent discoveries or a measure of the writer's first proposal.

D.3 A subsequent development window

Figure 14 pairs the cumulative program-event count with candidate gaps to the recorded-best utility. Both panels cover

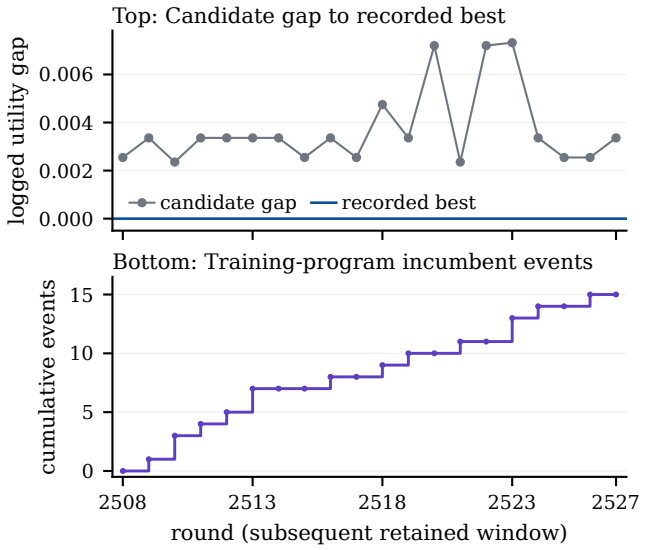


Figure 14: **Candidate gaps and training-program events in a subsequent window.** Top: recorded best minus candidate validation pool utility, paired within each completed round; zero marks the recorded best. Bottom: cumulative journaled strict training-program incumbent events, reset for this window. Both panels cover the same 20 completed summaries. The summaries mark no adapter admission. Repeated development comparisons do not establish independent first-proposal improvement.

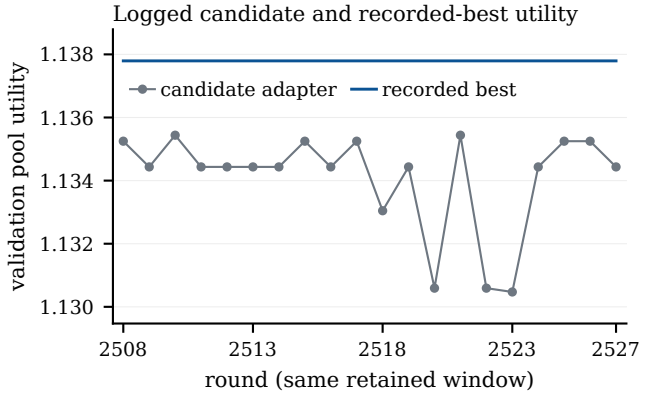


Figure 15: **Logged utility in the same development window.** The same 20 completed summaries from rounds 2508–2527 as in Figure 14. The recorded-best line is labeled round 882 in the logs; these summaries record no adapter admission. This alternate view adds no observations or independent first-proposal comparison.

rounds 2508–2527. Appendix D.10 summarizes the retained windows. Each gap compares utilities logged in one completed round.

D.4 Utility on its original scale

Figure 15 shows candidate and recorded-best validation pool utility on their original scale for the same summaries as Figure 14. It adds no observations. The best-round label is the log's recorded identity, not the first round of strict gain.

D.5 Completed-round rates

Figure 16 reuses the 20 summaries from rounds 2508–2527 in Appendix D.3. Each step divides one completed round, or its strict training-program event count, by that round's logged wall time. Its width is the corresponding duration. Dashed lines divide the window's total work or events by its total time; they do not average the per-round rates.

Time includes logged qualification waits and omits gaps outside completed rounds. These rates describe development activity, not deployment throughput. None of these summaries records an adapter admission.

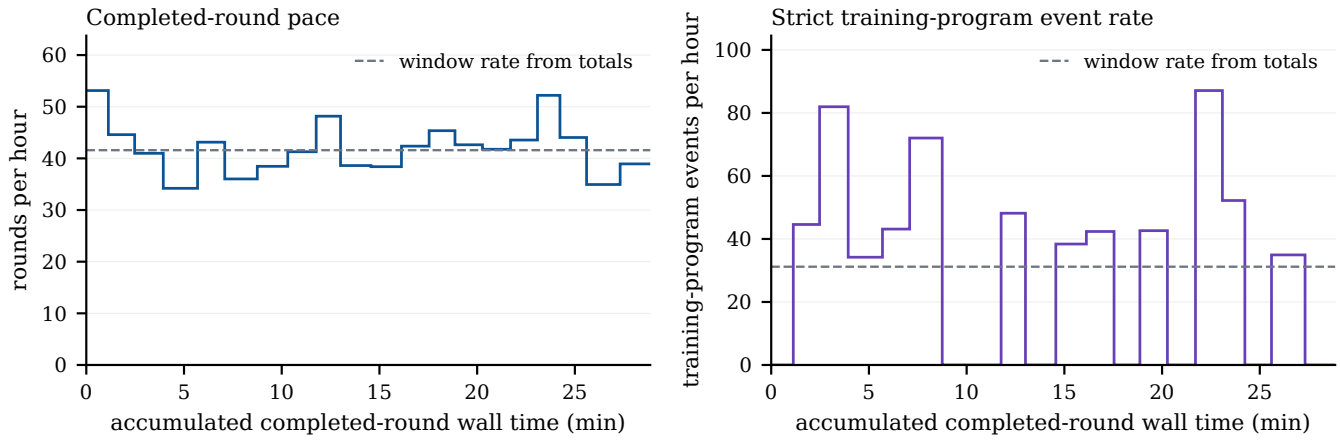


Figure 16: **Work and training-program event rates in the subsequent window.** Left: completed rounds per hour. Right: strict training-program incumbent events per hour. Each step is an average over one completed round's logged wall time; dashed lines are rates from whole-window totals. Both panels use the same 20 summaries as Figure 14, with no new observations. Program events do not count adapter admissions.

D.6 Adapter utility and completed-round throughput

Figure 8 extends the retained operating record with 20 later completed rounds, 2696–2715, captured on November 1, 2026 at 06:54 UTC. The summary and admission journals agree on every candidate utility, recorded best, acceptance flag and reported best-round marker in this window. The source retains no adapter admission here.

Throughput counts distinct training contexts within each round and divides by that round's full logged wall time. These within-round counts do not establish the number of distinct tasks across rounds. This rate has a different denominator from training presentations per training second.

D.7 Recorded-best trajectory and cumulative acceptance rate

Figure 17 provides a companion view of the 1,889 consecutive admission rounds, 775–2663, plotted in Figure 18. It adds no new observations. The left panel isolates the post-decision recorded-best validation pool utility step curve across the two strict improvements (rounds 785 and 859, reaching 1.1376 and 1.1378 from the initial 1.1074) and the two accepted ties (rounds 782 and 882). The right panel tracks the cumulative share of elapsed rounds accepted up to each round: the cumulative rate reaches 18.18% at round 785 (2/11 rounds), stands at 3.70% at the last accepted tie (4/108 at round 882), and declines across the 1,781 subsequent rejected rounds to 0.21% (4/1,889) by round 2663.

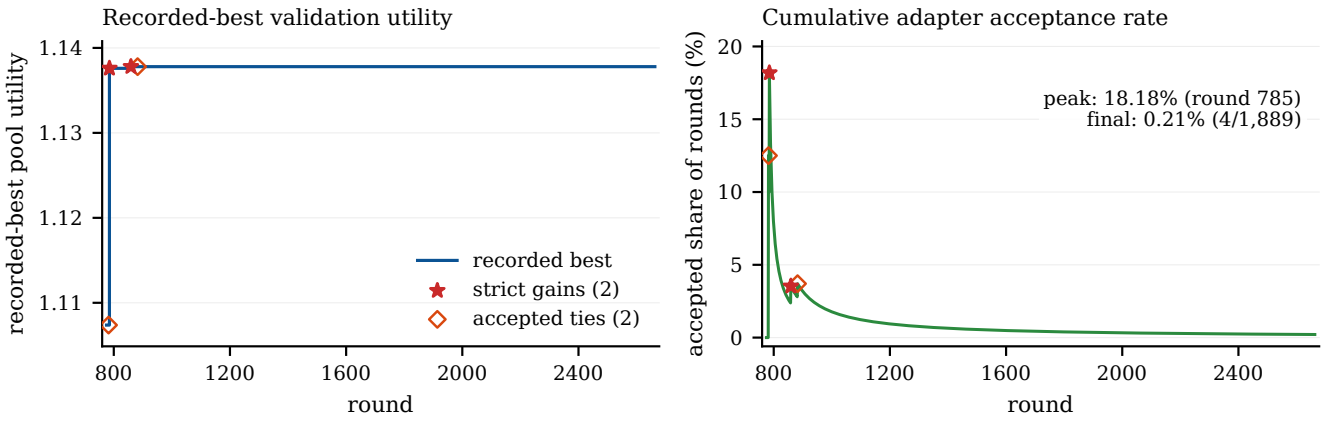


Figure 17: **Recorded-best validation utility and cumulative adapter acceptance rate over rounds.** Both panels use the same 1,889 consecutive admission rounds, 775–2663, as Figure 18, adding no observations. Left: post-decision recorded-best validation pool utility, with stars at strict gains (rounds 785 and 859) and hollow diamonds at accepted ties (rounds 782 and 882). Right: cumulative accepted share of elapsed rounds (%), which peaks at 18.18% at round 785 and falls to 0.21% (4/1,889) across the subsequent 1,781 rejected rounds. These are development admission aggregates, not independent first-proposal comparisons.

D.8 Promotions per hour and validation trajectory

Figure 7 pairs per-round and rolling rates of strict training-program promotions with candidate and recorded-best adapter validation utility. The 20 consecutive completed rounds, 2644–2663, are also shown in Figure 25.

D.9 Latest retained admission trace and pre-decision margins

The full admission chronology and counts are reported once in Section 6.8. This diagnostic uses the same retained prefix and classifies decisions from unrounded logged values.

Figure 6 decomposes the same 2,164 development records by their unrounded pre-decision validation margin, $\Delta_r = u_{\text{cand},r} - u_{\text{prev},r}$. The three strict gains and three exact ties are unchanged from the earlier prefix; the default rule accepts $\Delta_r \geq 0$. Among the 2,158 rejected rounds, 26 (1.20%) fall within $[-10^{-4}, 0)$ of a tie, 49 lie in $[-10^{-3}, -10^{-4})$, and 2,083 (96.52%) fall below -10^{-3}. The closest rejected margin is $-0.000{,}043{,}383{,}808{,}648{,}5$ at round 803; the median is -0.003360. These margins reuse the admission record; exact source, adapter and population binding remains pending in Section 6.8.

D.10 Earlier admission windows and historical search

The main text selects four views of admission decisions and operating activity. The additional figures here retain the earlier windows and alternative summaries, with their original captions and observations. Figure 18 covers the earlier admission prefix; Figures 19 and 20 resolve a smaller set of retained summaries. Gaps between windows are not filled or treated as observations.

Lower predicted cost is preferred. Logged utility averages the selected programs' recorded outcomes; it is not the adapter's cost prediction. For new proposals, the caller uses predicted costs and an exploration setting to select programs

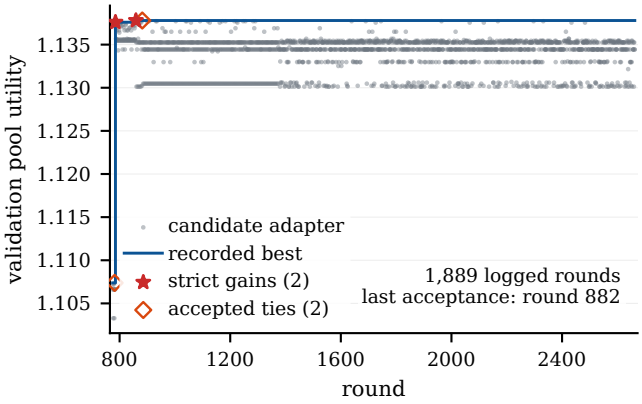


Figure 18: **Adapter validation and admissions.** The 1,889-round window, 775–2663: grey candidate utility, blue recorded best; stars mark strict gains, hollow diamonds accepted ties. Appendix D.9 extends through round 2938.

for exact evaluation. The referee still decides validity and improvement. Logging an evaluation does not itself add the program to the adapter's training pool.

Figure 18 shows the earlier 775–2663 prefix of the unified ledger in Section 6.8; Appendix D.7 provides its acceptance-rate view. Comparisons across harnesses require matched data, normalization and scoring protocols.

The older detailed summaries cover only 25 of those rounds. In that subset, training utility exceeds validation utility by 0.282–0.290 (Figure 20). This is a difference between reported aggregates on separate training and validation contexts; it does not isolate a cause or establish behavior in the missing summaries. The late validation plateau is evidence to explain, rather than proof of useful transfer. Encoder integrity and parameter updates require their own receipts; the plotted utility tables do not establish either. Figure 19 shows the loop's operating measurements from the earlier retained window.

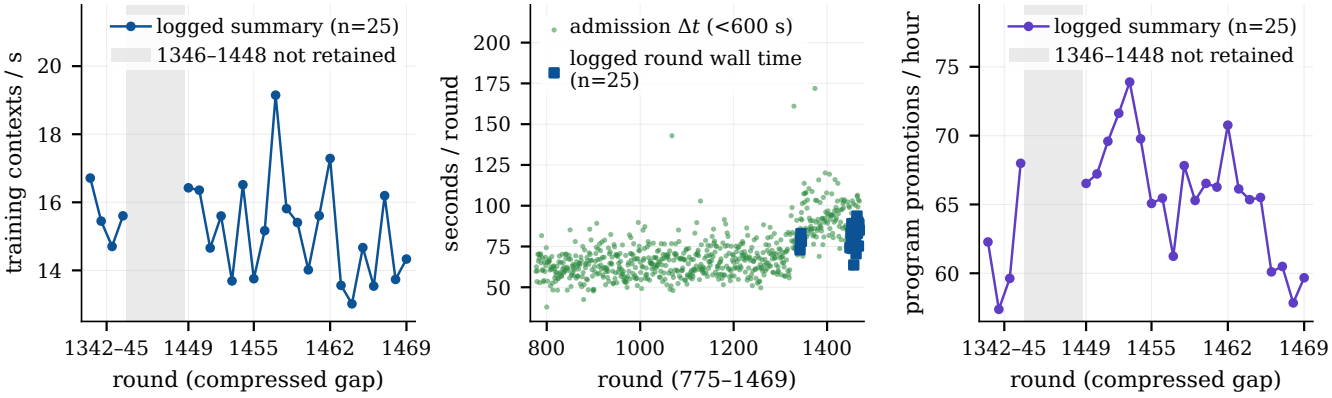


Figure 19: **Operating measurements from earlier retained windows.** Left/right: 25 summaries spanning rounds 1342–1469. Middle: intervals between logged admission timestamps within 775–1469. Green: positive intervals under 600 s. Blue squares: 25 logged round wall times. Left: distinct training contexts divided by completed-round wall time. Right: strict improvements to training-program incumbents per hour, using accumulated wall time over up to twenty completed rounds in the reporting process, including qualification waits. Program events and adapter admissions count different outcomes. These are earlier operating windows within Figure 18, not one-proposal deployment accuracy or cost.

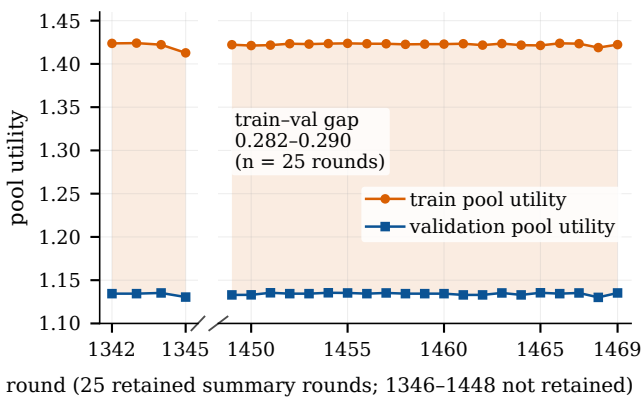


Figure 20: **Train and validation aggregates on retained summaries.** The 25 summaries span rounds 1342–1469; a broken axis separates the observed blocks. Intervening summaries are not retained; the utility gap covers only these 25 summaries.

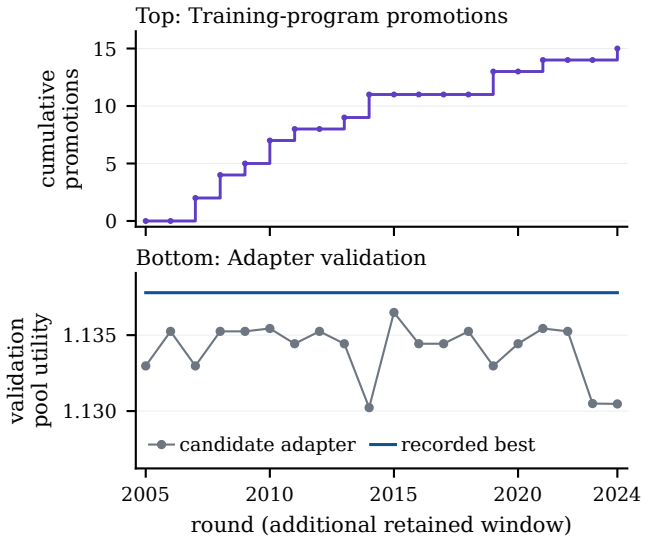


Figure 21: **Program promotions and adapter utility in an additional window.** Top: cumulative logged strict improvements to training-program incumbents, reset to zero at the start of this window. Bottom: candidate adapter validation utility and the recorded best. Both panels use the same 20 summaries from rounds 2005–2024. These are development measurements; neither the event count nor the utility curve establishes first-proposal improvement.

Retained operating windows. The five 20-round program-event slices contain 15 events in rounds 2005–2024, 10 in 2082–2101, 22 in 2343–2362, 15 in 2508–2527 and 21 in 2644–2663. Each records zero adapter admissions. The later 20-round throughput slice, 2696–2715, also records none; it reports contexts and wall time rather than a program-event total. Figures 14, 21, 22, 24 and 25 show the event slices. Completed-time, event-distribution and rate summaries appear in Appendices D.1, D.2, D.5, D.6 and D.8. These are separate retained windows, not a matched learning curve; unobserved rounds are not filled. Program-event rates include the logged work and qualification waits and do not measure one-proposal deployment.

For the adapter record in Figure 18, Figure 23 summarizes candidate gaps to each round's recorded best. Each retained round contributes once; the curve does not count independent tasks or distinct programs.

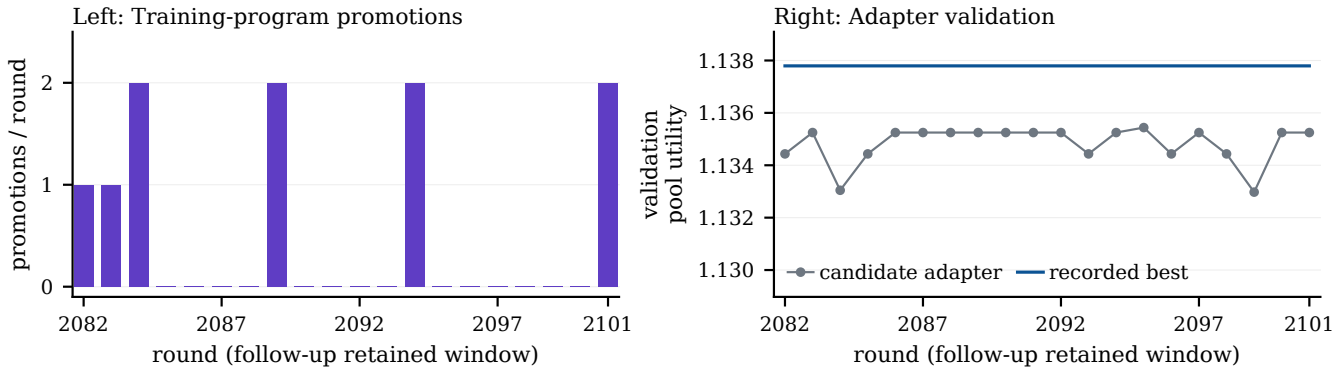


Figure 22: **Program events and adapter validation in a follow-up window.** Left: journaled strict training-program incumbent improvements per round. Right: candidate adapter validation pool utility and the recorded best. Both panels cover the same 20 consecutive summary rounds, 2082–2101. The summaries mark no adapter admission. Independent first-proposal and transfer tests remain required.

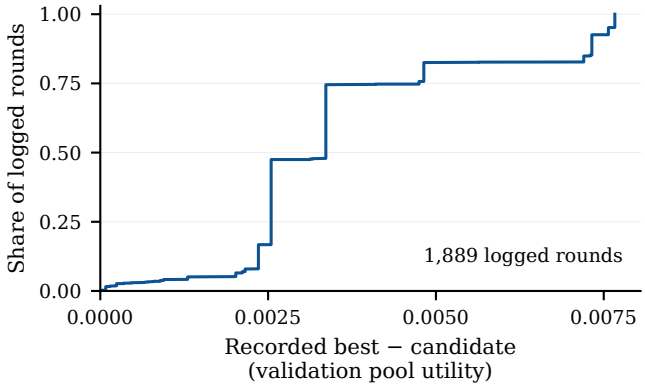


Figure 23: **Candidate gaps to the recorded best.** Cumulative share of the same 1,889 admission rounds as Figure 18, against recorded best minus candidate validation utility. The best is logged after admission, so zero includes strict gains and ties. Repeated adaptive rounds are not independent trials.

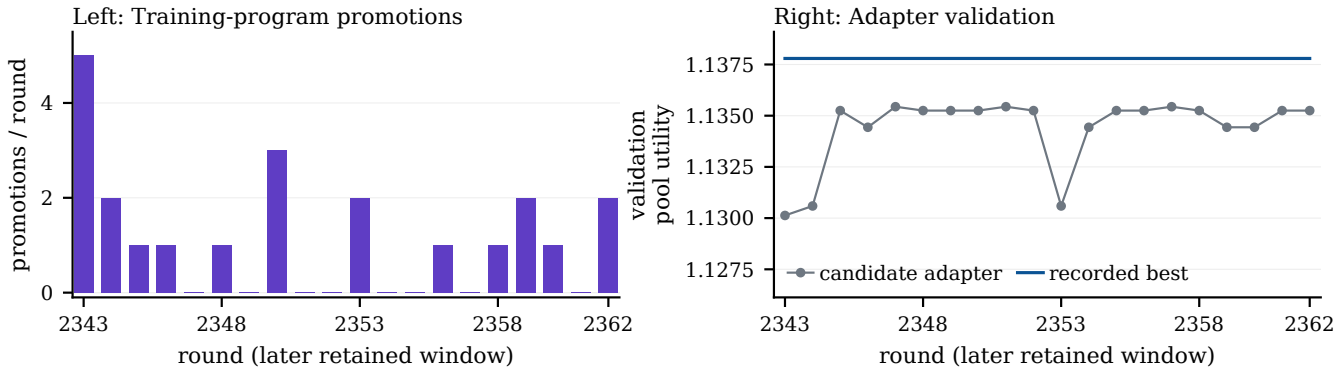


Figure 24: **Continued program events and adapter validation.** Left: journaled strict training-program incumbent improvements per round. Right: candidate adapter validation pool utility and the recorded best. Both panels cover all 20 consecutive rounds, 2343–2362, in this retained summary window. The summaries mark no adapter admission. Independent first-proposal comparisons remain required.

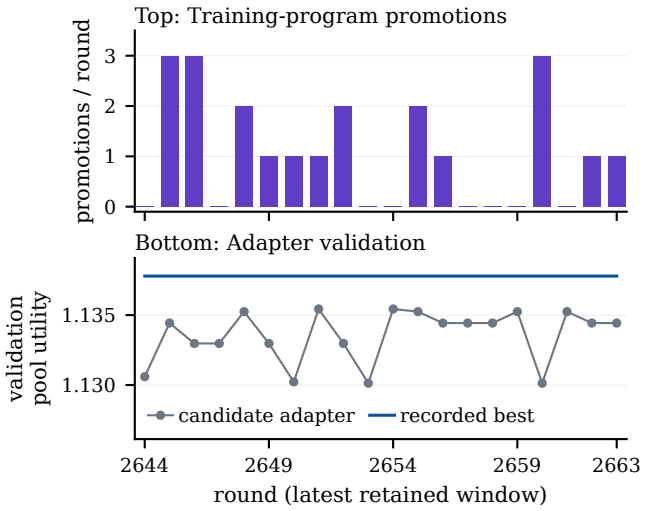


Figure 25: **Program-incumbent events and validation summaries.** Top: logged strict improvements to training-program incumbents per round. Bottom: candidate adapter validation pool utility. Both panels cover the same 20 consecutive rounds, 2644–2663. These are development measurements; independent first-proposal comparisons remain required.

Historical GP target scores provide a separate view in Figure 26. The 159 repeated dataset/cut/arm groups cover 55 distinct dataset/cut pairs; the 64 groups with lower final scores cover 23 pairs. Shared incumbent and teacher seeds make the arms dependent. These retained development outcomes do not establish that learning improved the proposal policy.

Figure 27 follows one historical dataset/cut, chosen after inspection for the largest policy-seeded relative improvement in the preceding comparison set. Every reported generation is shown, without applying a cumulative minimum. All three arms finish at the same value; the policy-seeded arm first records it one generation after the controls. Shared seeds prevent treating their matching outcomes as independent discoveries.

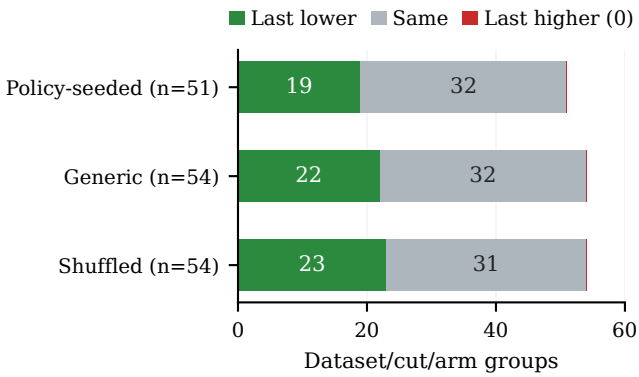


Figure 26: **Historical GP target progress.** First-to-last reported best development RMSE at 1,000 evaluations per search, grouped by search arm. Lower is better; unchanged scores are included. The arms shared incumbent and teacher seeds, and elapsed time varied. These counts do not measure independent discoveries or a learned-policy advantage.

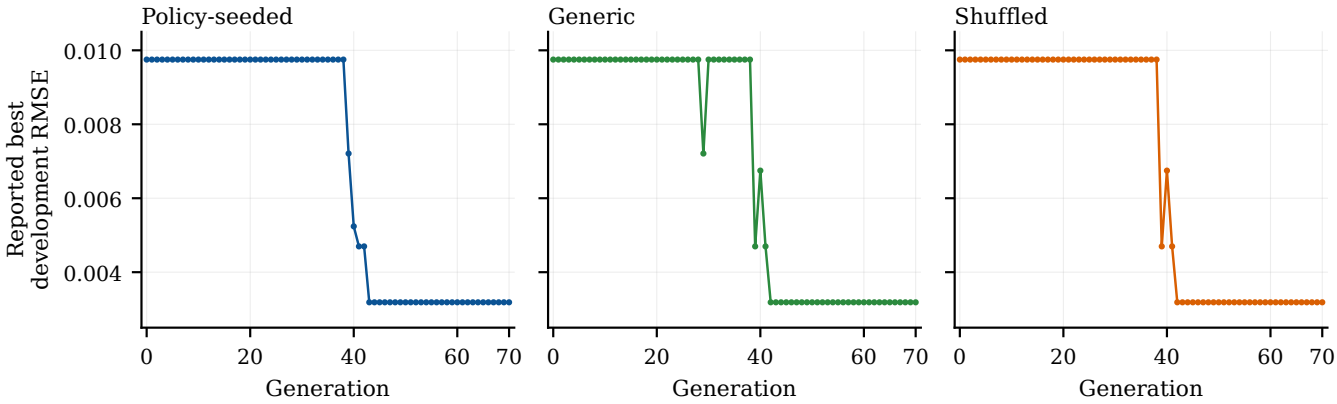


Figure 27: **Post-hoc GP target trajectory.** All 71 generations per arm for one historical dataset/cut, at 1,000 evaluations per search. RMSE is in historical evaluator units; lower is better. The arms shared incumbent and teacher seeds, and elapsed time varied. This selected development example does not establish a learned-policy advantage.

References

[1] T. Anthony, Z. Tian, and D. Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, volume 30, pages 5360–5370, 2017. doi: 10.48550/arXiv.1705.08439. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/d8e1344e27a5b08cdfd5d027d9b8d6de-Paper.pdf.

[2] P. Azoulay, C. Fons-Rosen, and J. S. Graff Zivin. Does science advance one funeral at a time? *American Economic Review*, 109(8):2889–2920, 2019. doi: 10.1257/aer.20161574.

[3] L. Bailey, K. Wen, K. Dong, T. Hashimoto, and T. Ma. Scaling self-play with self-guidance. arXiv:2604.20209, April 2026. URL https://arxiv.org/abs/2604.20209.

[4] D. Balduzzi, K. Tuyls, J. Pérolat, and T. Graepel. Re-evaluating evaluation. In *Advances in Neural Information Processing Systems*, volume 31, pages 3268–3279. Curran Associates, 2018. doi: 10.48550/arXiv.1806.02643. URL https://arxiv.org/abs/1806.02643.

[5] M. Balog, A. L. Gaunt, M. Brockschmidt, S. Nowozin, and D. Tarlow. DeepCoder: Learning to write programs. In *International Conference on Learning Representations (ICLR)*, 2017. doi: 10.48550/arXiv.1611.01989. URL https://arxiv.org/abs/1611.01989.

[6] R. Bellman. *Dynamic Programming*. Princeton University Press, Princeton, NJ, 1957. doi: 10.1515/9781400835386.

[7] J. Bennett and S. Lanning. The Netflix Prize. In *Proceedings of KDD Cup and Workshop 2007*, 2007. URL https://www.cs.uic.edu/~liub/KDD-cup-2007/proceedings/The-Netflix-Prize-Bennett.pdf.

[8] N. Bloom, C. I. Jones, J. Van Reenen, and M. Webb. Are ideas getting harder to find? *American Economic Review*, 110 (4):1104–1144, 2020. doi: 10.1257/aer.20180338.

[9] M. A. Boden. *The Creative Mind: Myths and Mechanisms*. Weidenfeld & Nicolson, London, 1990. doi: 10.4324/9780203508527.

[10] M. A. Boden. Creativity and artificial intelligence. *Artificial Intelligence*, 103(1–2):347–356, 1998. doi: 10.1016/S0004-3702(98)00055-1.

[11] J. L. Borges. La biblioteca de Babel. In *El jardín de senderos que se bifurcan*. Sur, Buenos Aires, 1941. Printed 30 December 1941, issued 1942 (LCCN 43002084). English translation by James E. Irby, "The Library of Babel", in *Labyrinths*, New Directions, 1962.

[12] R. A. Bradley and M. E. Terry. Rank analysis of incomplete block designs: I. The method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952. doi: 10.1093/biomet/39.3-4.324.

[13] R. Brandom. What happens when AI starts building itself? TechCrunch, 14 May 2026. URL https://techcrunch.com/2026/05/14/what-happens-when-ai-starts-building-itself/.

[14] T. Brooks, B. Peebles, C. Holmes, W. DePue, Y. Guo, L. Jing, D. Schnurr, J. Taylor, T. Luhman, E. Luhman, C. Ng, R. Wang, and A. Ramesh. Video generation models as world simulators. OpenAI technical report, February 2024. URL https://openai.com/index/video-generation-models-as-world-simulators/.

[15] D. T. Campbell. Blind variation and selective retention in creative thought as in other knowledge processes. *Psychological Review*, 67(6):380–400, 1960. doi: 10.1037/h0040373.

[16] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai, and H. Li. Learning to rank: From pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning (ICML)*, pages 129–136, 2007. doi: 10.1145/1273496.1273513.

[17] G. C. Cawley and N. L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70): 2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[18] W. G. Chase and H. A. Simon. Perception in chess. *Cognitive Psychology*, 4(1):55–81, 1973. doi: 10.1016/0010-0285(73)90004-2.

[19] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. W. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. A. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders,

C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. arXiv:2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

[20] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, and Q. V. Le. Symbolic discovery of optimization algorithms. In *Advances in Neural Information Processing Systems*, volume 36, pages 49205–49233, 2023. doi: 10.48550/arXiv.2302.06675. URL https://arxiv.org/abs/2302.06675.

[21] Z. Chen, Y. Deng, H. Yuan, K. Ji, and Q. Gu. Self-play fine-tuning converts weak language models to strong language models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 6621–6642, 2024. doi: 10.48550/arXiv.2401.01335. URL https://arxiv.org/abs/2401.01335.

[22] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, volume 30, pages 4299–4307, 2017. doi: 10.48550/arXiv.1706.03741. URL https://arxiv.org/abs/1706.03741.

[23] S. Cooper, F. Khatib, A. Treuille, J. Barbero, J. Lee, M. Beenen, A. Leaver-Fay, D. Baker, Z. Popović, and Foldit Players. Predicting protein structures with a multiplayer online game. *Nature*, 466(7307):756–760, 2010. doi: 10.1038/nature09304.

[24] H. Dai, Y. Tian, B. Dai, S. Skiena, and L. Song. Syntax-directed variational autoencoder for structured data. In *International Conference on Learning Representations (ICLR)*, 2018. doi: 10.48550/arXiv.1802.08786. URL https://arxiv.org/abs/1802.08786.

[25] C. Darwin. *On the Origin of Species by Means of Natural Selection, or the Preservation of Favoured Races in the Struggle for Life*. John Murray, London, 1859. doi: 10.5962/bhl.title.68064. URL http://darwin-online.org.uk/content/frameset?itemID=F373&viewtype=text&pageseq=1.

[26] A. D. de Groot. *Thought and Choice in Chess*. Mouton, The Hague, 1965. doi: 10.5117/9789053569986.

[27] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, et al. DeepSeek-V3 technical report. arXiv:2412.19437, 2024. URL https://arxiv.org/abs/2412.19437.

[28] K. Dong and T. Ma. STP: Self-play LLM theorem provers with iterative conjecturing and proving. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 14114–14136, 2025. doi: 10.48550/arXiv.2502.00212. URL https://proceedings.mlr.press/v267/dong25h.html.

[29] D. Donoho. 50 years of data science. *Journal of Computational and Graphical Statistics*, 26(4):745–766, 2017. doi: 10.1080/10618600.2017.1384734.

[30] S. Du, X. Yan, J. Shi, Z. Cao, S. Feng, Z. Liang, B. Sun, T. Peng, Y. Zhou, X. Li, J. Zhou, L. He, B. Zhang, and L. Bai. MLEvolve: A self-evolving framework for automated machine learning algorithm discovery. arXiv:2606.06473, June 2026. URL https://arxiv.org/abs/2606.06473.

[31] K. Ellis, C. Wong, M. Nye, M. Sablé-Meyer, L. Morales, L. Hewitt, L. Cary, A. Solar-Lezama, and J. B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 835–850, 2021. doi: 10.1145/3453483.3454080.

[32] A. E. Elo. *The Rating of Chessplayers, Past and Present*. Arco, New York, 1978.

[33] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, D. Silver, D. Hassabis, and P. Kohli. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022. doi: 10.1038/s41586-022-05172-4.

[34] M. Fey, V. Kocijan, F. Lopez, J. E. Lenssen, and J. Leskovec. KumoRFM: A relational foundation model. Kumo Technical Report, May 2025. URL https://docs.nvidia.com/sdgm/research/concepts-models/kumorfm-paper.

[35] R. M. Friedberg. A learning machine: Part I. *IBM Journal of Research and Development*, 2(1):2–13, 1958. doi: 10.1147/rd.21.0002.

[36] I. J. Good. Speculations concerning the first ultraintelligent machine. In F. L. Alt and M. Rubinoff, editors, *Advances in Computers*, volume 6, pages 31–88. Academic Press, 1966. doi: 10.1016/S0065-2458(08)60418-0.

[37] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, P. Sirkovic, A. Myaskovsky, G. Glowaty, F. Weissenberger, A. Orlandi, D. Popovici, A. Palepu, K. Rong, R. Tanno, K. Saab, F. Zhang, J. Blum, A. Carroll, K. Kulkarni, N. Tomasev, D. Zverinski, I. Rendulic, E. Vedadi, F. Hasler, L. Rimanic, M. Boia, I. Budiselic, B. Feinstein, M. Bellaiche, T. Sheffer, J. Freyberg, J. Ratcliff, O. Bertolli, K. Chou, A. Hassidim, B. Gokturk, A. Vahdat, Y. Guan, V. Dhillon, E. D. Vaishnav, B. Lee, T. R. D. Costa, J. R. Penadés, G. Peltz, Y. Matias, J. Manyika, D. Hassabis, Y. Xu, P. Kohli, A. Pawlosky, A. Karthikesalingam, and V. Natarajan. Accelerating scientific discovery with Co-Scientist. *Nature*, 655(8122): 487–496, 2026. doi: 10.1038/s41586-026-10644-y. Preprint: arXiv:2502.18864 (2025), "Towards an AI co-scientist".

[38] S. Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 317–330, 2011. doi: 10.1145/1926385.1926423.

[39] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081): 633–638, 2025. doi: 10.1038/s41586-025-09422-z.

[40] R. W. Hamming. You and your research. Bell Communications Research Colloquium Seminar, Morris Research and Engineering Center, Morristown, NJ, 1986. URL https://hdl.handle.net/10945/37504. Talk given 7 March 1986; transcribed by J. F. Kaiser.

[41] D. Hassabis and A. Kantrowitz. Google DeepMind CEO Demis Hassabis: The path to AGI, LLM creativity, and Google smart glasses. Big Technology Podcast, 23 January

2025. URL https://www.bigtechnology.com/p/google-deepmind-ceo-demis-hassabis.

[42] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022. doi: 10.48550/arXiv.2203.15556. URL https://arxiv.org/abs/2203.15556.

[43] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975. doi: 10.7551/mitpress/1090.001.0001.

[44] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, 2025. doi: 10.1038/s41586-024-08328-6.

[45] T. Hubert, R. Mehta, L. Sartran, M. Z. Horváth, G. Žužić, E. Wieser, A. Huang, J. Schrittwieser, Y. Schroecker, H. Masoom, O. Bertolli, T. Zahavy, A. Mandhane, J. Yung, I. Beloshapka, B. Ibarz, V. Veeriah, L. Yu, O. Nash, P. Lezeau, S. Mercuri, C. Sönne, B. Mehta, A. Davies, D. Zheng, F. Pedregosa, Y. Li, I. von Glehn, M. Rowland, S. Albanie, A. Velingker, S. Schmitt, E. Lockhart, E. Hughes, H. Michalewski, N. Sonnerat, D. Hassabis, P. Kohli, and D. Silver. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y. Published online 12 November 2025.

[46] B. F. Jones. The burden of knowledge and the "death of the Renaissance man": Is innovation getting harder? *Review of Economic Studies*, 76(1):283–317, 2009. doi: 10.1111/j.1467-937X.2008.00531.x.

[47] J. Jumper, R. Evans, A. Pritzel, T. Green, M. Figurnov, O. Ronneberger, K. Tunyasuvunakool, R. Bates, A. Žídek, A. Potapenko, A. Bridgland, C. Meyer, S. A. A. Kohl, A. J. Ballard, A. Cowie, B. Romera-Paredes, S. Nikolov, R. Jain, J. Adler, T. Back, S. Petersen, D. Reiman, E. Clancy, M. Zielinski, M. Steinegger, M. Pacholska, T. Berghammer, S. Bodenstein, D. Silver, O. Vinyals, A. W. Senior, K. Kavukcuoglu, P. Kohli, and D. Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596 (7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

[48] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. arXiv:2001.08361, 2020. URL https://arxiv.org/abs/2001.08361.

[49] A. Karpathy. `autoresearch`: AI agents running research on single-GPU nanochat training automatically. GitHub repository, 6 March 2026. URL https://github.com/karpathy/autoresearch.

[50] F. Khatib, F. DiMaio, Foldit Contenders Group, Foldit Void Crushers Group, S. Cooper, M. Kazmierczyk, M. Gilski, S. Krzywda, H. Zábranská, I. Pichová, J. Thompson, Z. Popović, M. Jaskolski, and D. Baker. Crystal structure of a monomeric retroviral protease solved by protein folding game players. *Nature Structural & Molecular Biology*, 18(10): 1175–1177, 2011. doi: 10.1038/nsmb.2119.

[51] Kimi Team. Kimi K2: Open agentic intelligence. arXiv:2507.20534, 2025. URL https://arxiv.org/abs/2507.20534.

[52] A. Koestler. *The Act of Creation*. Hutchinson, London, 1964.

[53] J. R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, 1992. URL https://mitpress.mit.edu/9780262111706/genetic-programming/.

[54] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, pages 1097–1105, 2012. doi: 10.1145/3065386. URL https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks. Republished in *Communications of the ACM*, 60(6):84–90, 2017.

[55] T. S. Kuhn. *The Structure of Scientific Revolutions*. University of Chicago Press, Chicago, 1962. doi: 10.7208/chicago/9780226458144.001.0001.

[56] M. J. Kusner, B. Paige, and J. M. Hernández-Lobato. Grammar variational autoencoder. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1945–1954, 2017. doi: 10.48550/arXiv.1703.01925. URL https://proceedings.mlr.press/v70/kusner17a.html.

[57] T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, Cambridge, 2020. doi: 10.1017/9781108571401.

[58] Y. LeCun and A. Kantrowitz. Why can't AI make its own discoveries? — with Yann LeCun. Big Technology Podcast, 19 March 2025. URL https://podscripts.co/podcasts/big-technology-podcast/why-cant-ai-make-its-own-discoveries-with-yann-lecun.

[59] Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.

[60] J. Lee, W. Kladwang, M. Lee, D. Cantu, M. Azizyan, H. Kim, A. Limpaecher, S. Yoon, A. Treuille, R. Das, and EteRNA Participants. RNA design rules from a massive open laboratory. *Proceedings of the National Academy of Sciences*, 111(6):2122–2127, 2014. doi: 10.1073/pnas.1313039111.

[61] Y.-J. Lee, S. Kim, M. Kang, A. C. L. Chuen, Z. Chen, S. Han, T. Jung, and D. Kang. Evolution Fine-Tuning: Learning to discover across 371 optimization tasks. arXiv:2606.29082, June 2026. URL https://arxiv.org/abs/2606.29082.

[62] J. Lehman and K. O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011. doi: 10.1162/EVCO_a_00025.

[63] D. B. Lenat. *AM: An Artificial Intelligence Approach to Discovery in Mathematics as Heuristic Search*. PhD thesis, Stanford University, 1976. Stanford AI Lab Memo AIM-286; report STAN-CS-76-570.

[64] D. B. Lenat. EURISKO: A program that learns new heuristics and domain concepts. The nature of heuristics III: Program design and results. *Artificial Intelligence*, 21(1–2):61–98, 1983. doi: 10.1016/S0004-3702(83)80005-8.

[65] C. Lu, C. Lu, R. T. Lange, Y. Yamada, S. Hu, J. Foerster, D. Ha, and J. Clune. Towards end-to-end automation of AI research. *Nature*, 651(8107):914–919, 2026. doi: 10.1038/s41586-026-10265-5. Preprint: arXiv:2408.06292 (2024), "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery".

[66] D. J. Mankowitz, A. Michi, A. Zhernov, M. Gelmi, M. Selvi, C. Paduraru, E. Leurent, S. Iqbal, J.-B. Lespiau, A. Ahern, T. Köppe, K. Millikin, S. Gaffney, S. Elster, J. Broshear, C. Gamble, K. Milan, R. Tung, M. Hwang, T. Cemgil, M. Barekatain, Y. Li, A. Mandhane, T. Hubert, J. Schrittwieser, D. Hassabis, P. Kohli, M. Riedmiller, O. Vinyals, and D. Silver. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618 (7964):257–263, 2023. doi: 10.1038/s41586-023-06004-9.

[67] H. Massalin. Superoptimizer: A look at the smallest program. In *Proceedings of the 2nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS II)*, pages 122–126, 1987. doi: 10.1145/36206.36194.

[68] A. Merchant, S. Batzner, S. S. Schoenholz, M. Aykol, G. Cheon, and E. D. Cubuk. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85, 2023. doi: 10.1038/s41586-023-06735-9.

[69] C. Metz. In two moves, AlphaGo and Lee Sedol redefined the future. *Wired*, 16 March 2016. URL https://www.wired.com/2016/03/two-moves-alphago-lee-sedol-redefined-future/.

[70] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.

[71] D. J. Montana. Strongly typed genetic programming. *Evolutionary Computation*, 3(2):199–230, 1995. doi: 10.1162/evco.1995.3.2.199.

[72] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015. URL https://arxiv.org/abs/1504.04909.

[73] A. Müller, C. Curino, and R. Ramakrishnan. MotherNet: Fast training and inference via hyper-network transformers. In *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2312.08598. URL https://arxiv.org/abs/2312.08598.

[74] J. Nielsen and J. D. Keasling. Engineering cellular metabolism. *Cell*, 164(6):1185–1197, 2016. doi: 10.1016/j.cell.2016.02.004.

[75] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131, 2025. URL https://arxiv.org/abs/2506.13131.

[76] NVIDIA. NVIDIA H100 tensor core GPU: Specifications. NVIDIA Data Center Documentation, 2026. URL https://www.nvidia.com/en-us/data-center/h100/.

[77] J. Oh, G. Farquhar, I. Kemaev, D. A. Calian, M. Hessel, L. Zintgraf, S. Singh, H. van Hasselt, and D. Silver. Discovering state-of-the-art reinforcement learning algorithms. *Nature*, 648(8093):312–319, 2025. doi: 10.1038/s41586-025-09761-x.

[78] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744, 2022. doi: 10.48550/arXiv.2203.02155. URL https://arxiv.org/abs/2203.02155.

[79] M. Park, E. Leahey, and R. J. Funk. Papers and patents are becoming less disruptive over time. *Nature*, 613(7942): 138–144, 2023. doi: 10.1038/s41586-022-05543-x.

[80] D. Patel and D. Amodei. Dario Amodei (Anthropic CEO) — the hidden pattern behind every AI breakthrough. Dwarkesh Podcast, 8 August 2023. URL https://www.dwarkesh.com/p/dario-amodei.

[81] C. S. Peirce. *Collected Papers of Charles Sanders Peirce, Volume V: Pragmatism and Pragmaticism*. Harvard University Press, Cambridge, MA, 1934. Edited by Charles Hartshorne and Paul Weiss.

[82] B. Pfahringer, H. Bensusan, and C. Giraud-Carrier. Meta-learning by landmarking various learning algorithms. In *Proceedings of the 17th International Conference on Machine Learning (ICML)*, pages 743–750. Morgan Kaufmann, 2000. URL https://ml.cms.waikato.ac.nz/publications/2000/landmarking_icml2000.pdf.

[83] H. Poincaré. Mathematical creation. *The Monist*, 20(3): 321–335, 1910. doi: 10.1093/monist/20.3.321. Translated by George Bruce Halsted.

[84] M. Polanyi. *The Tacit Dimension*. Doubleday, Garden City, NY, 1966.

[85] G. Pólya. *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, Princeton, NJ, 1945. doi: 10.1515/9781400828678.

[86] K. R. Popper. *The Logic of Scientific Discovery*. Hutchinson, London, 1959. doi: 10.4324/9780203994627.

[87] K. R. Popper. *Conjectures and Refutations: The Growth of Scientific Knowledge*. Routledge & Kegan Paul, London, 1963. doi: 10.4324/9780203538074.

[88] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, 2021. doi: 10.48550/arXiv.2103.00020. URL https://proceedings.mlr.press/v139/radford21a.html.

[89] E. Real, C. Liang, D. R. So, and Q. V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In H. Daumé III and A. Singh, editors, *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 8007–8019. PMLR, 2020. doi: 10.48550/arXiv.2003.03384. URL https://proceedings.mlr.press/v119/real20a.html.

[90] J. R. Rice. The algorithm selection problem. In M. Rubinoff and M. C. Yovits, editors, *Advances in Computers*, volume 15, pages 65–118. Academic Press, New York, 1976. doi: 10.1016/S0065-2458(08)60520-3.

[91] B. Romera-Paredes, M. Barekatain, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

[92] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3): 211–252, 2015. doi: 10.1007/s11263-015-0816-y.

[93] A. L. Samuel. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229, 1959. doi: 10.1147/rd.33.0210.

[94] J. Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. arXiv:cs/0309048, 2003. URL https://arxiv.org/abs/cs/0309048.

[95] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020. doi: 10.1038/s41586-020-03051-4.

[96] C. E. Shannon. Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950. doi: 10.1080/14786445008521796.

[97] M. Shin, J. Kim, B. van Opheusden, and T. L. Griffiths. Superhuman artificial intelligence can improve human decision-making by increasing novelty. *Proceedings of the National Academy of Sciences*, 120(12):e2214840120, 2023. doi: 10.1073/pnas.2214840120.

[98] C. Si, D. Yang, and T. Hashimoto. Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers. In *International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2409.04109. URL https://openreview.net/forum?id=M23dTGWCZy.

[99] C. Si, T. Hashimoto, and D. Yang. The ideation–execution gap: Execution outcomes of LLM-generated versus human research ideas. In *International Conference on Learning Representations (ICLR)*, 2026. doi: 10.48550/arXiv.2506.20803. URL https://openreview.net/forum?id=Fllp8l6Puy.

[100] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

[101] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017. doi: 10.1038/nature24270.

[102] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018. doi: 10.1126/science.aar6404. URL https://arxiv.org/abs/1712.01815. Earlier version: arXiv:1712.01815 (2017). Training statistics: supplementary Table S3.

[103] D. K. Simonton. Creativity and discovery as blind variation: Campbell's (1960) BVSR model after the half-century mark. *Review of General Psychology*, 15(2):158–174, 2011. doi: 10.1037/a0022912.

[104] R. Socher. AI's biggest breakthrough won't be a chatbot. Next Big Idea Club, Book Bites, 8 October 2026. URL https://nextbigideaclub.com/magazine/ais-biggest-breakthrough-wont-chatbot-bookbite/61534/.

[105] R. Socher. *The Eureka Machine: Why AI Is the Key to Unlocking a New Era of Scientific Discoveries*. PublicAffairs, New York, 2026. ISBN 9781541705708. URL https://www.hachettebookgroup.com/titles/richard-socher/the-eureka-machine/9781541705708/.

[106] J. M. Stokes, K. Yang, K. Swanson, W. Jin, A. Cubillos-Ruiz, N. M. Donghia, C. R. MacNair, S. French, L. A. Carfrae, Z. Bloom-Ackermann, V. M. Tran, A. Chiappino-Pepe, A. H. Badran, I. W. Andrews, E. J. Chory, G. M. Church, E. D. Brown, T. S. Jaakkola, R. Barzilay, and J. J. Collins. A deep learning approach to antibiotic discovery. *Cell*, 180(4): 688–702.e13, 2020. doi: 10.1016/j.cell.2020.01.021.

[107] A. Surina, A. Mansouri, L. Quaedvlieg, A. Seddas, M. Viazovska, E. Abbe, and C. Gulcehre. Algorithm discovery with LLMs: Evolutionary search meets reinforcement learning. arXiv:2504.05108, 2025. URL https://arxiv.org/abs/2504.05108.

[108] R. S. Sutton. The bitter lesson. Essay, 13 March 2019. URL http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

[109] G. Tesauro. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3):58–68, 1995. doi: 10.1145/203330.203343.

[110] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024. doi: 10.1038/s41586-023-06747-5.

[111] A. van den Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018. URL https://arxiv.org/abs/1807.03748.

[112] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, 2017. doi: 10.48550/arXiv.1706.03762. URL https://arxiv.org/abs/1706.03762.

[113] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka,

A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama, D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019. doi: 10.1038/s41586-019-1724-z.

[114] S. Von Arx, L. Chan, and B. Barnes. Recent frontier models are reward hacking. METR blog, 5 June 2025. URL https://metr.org/blog/2025-06-05-recent-reward-hacking/.

[115] G. Wallas. *The Art of Thought*. Jonathan Cape, London, 1926.

[116] K. Wiggers. Sakana walks back claims that its AI can dramatically speed up model training. TechCrunch, 21 February 2025. URL https://techcrunch.com/2025/02/21/sakana-walks-back-claims-that-its-ai-can-dramatically-speed-up-model-training/.

[117] T. Wolf. The Einstein AI model. Technical blog essay, February 2025. URL https://thomwolf.io/blog/scientific-ai.html.

[118] D. H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.

[119] M. Yuksekgonul, D. Koceja, X. Li, F. Bianchi, J. McCaleb, X. Wang, J. Kautz, Y. Choi, J. Zou, C. Guestrin, and Y. Sun. Learning to discover at test time. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. doi: 10.48550/arXiv.2601.16175. URL https://arxiv.org/abs/2601.16175.

[120] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai. Self-Taught Optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling (COLM)*, 2024. doi: 10.48550/arXiv.2310.02304. URL https://arxiv.org/abs/2310.02304.

[121] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune. Darwin Gödel Machine: Open-ended evolution of self-improving agents. arXiv:2505.22954, 2025. URL https://arxiv.org/abs/2505.22954.

[122] A. Zhao, Y. Wu, Y. Yue, T. Wu, Q. Xu, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang. Absolute Zero: Reinforced self-play reasoning with zero data. arXiv:2505.03335, 2025. URL https://arxiv.org/abs/2505.03335.

[123] Z. Zhong, I. Segal, I. Bercovich, S. Saxena, K. Zhang, and A. Raghunathan. Hardening agent benchmarks with adversarial hacker-fixer loops. arXiv:2606.08960, 2026. URL https://arxiv.org/abs/2606.08960.

[124] B. Zoph and Q. V. Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2017. doi: 10.48550/arXiv.1611.01578. URL https://arxiv.org/abs/1611.01578.